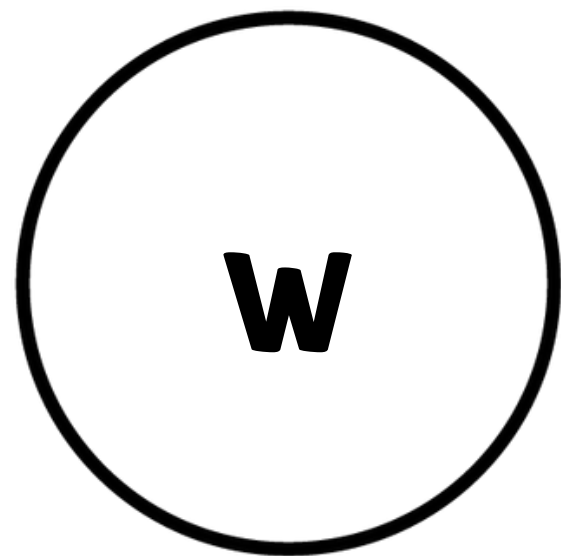




CLASE 6

DIJKSTRA

GRAFOS CON PESOS



NODOS

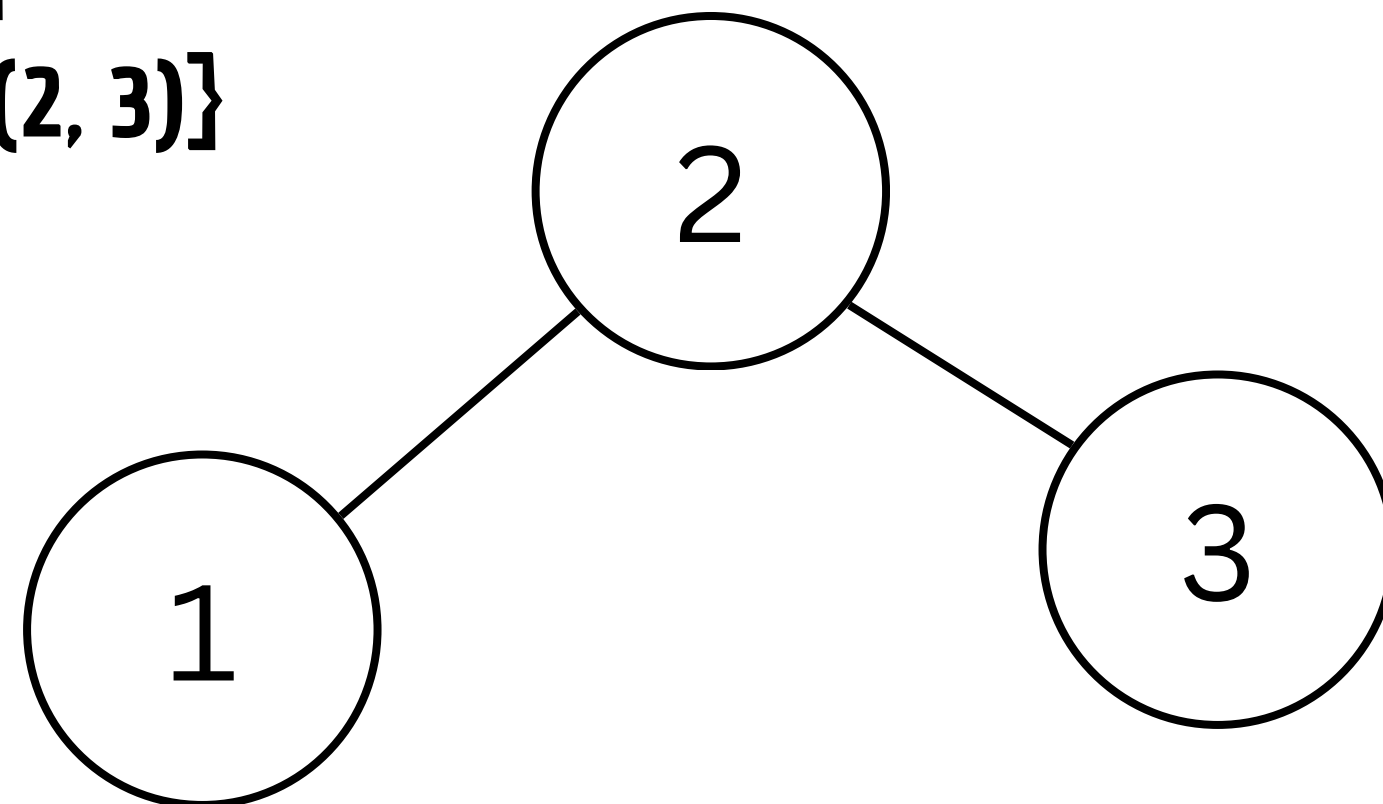


ARISTAS

ARISTAS CON PESOS

Formalmente un grafo es:

V = conjunto de nodos Ej: $\{1, 2, 3\}$
 E = conjunto de aristas Ej: $\{(1, 2), (2, 3)\}$



ARISTAS CON PESOS

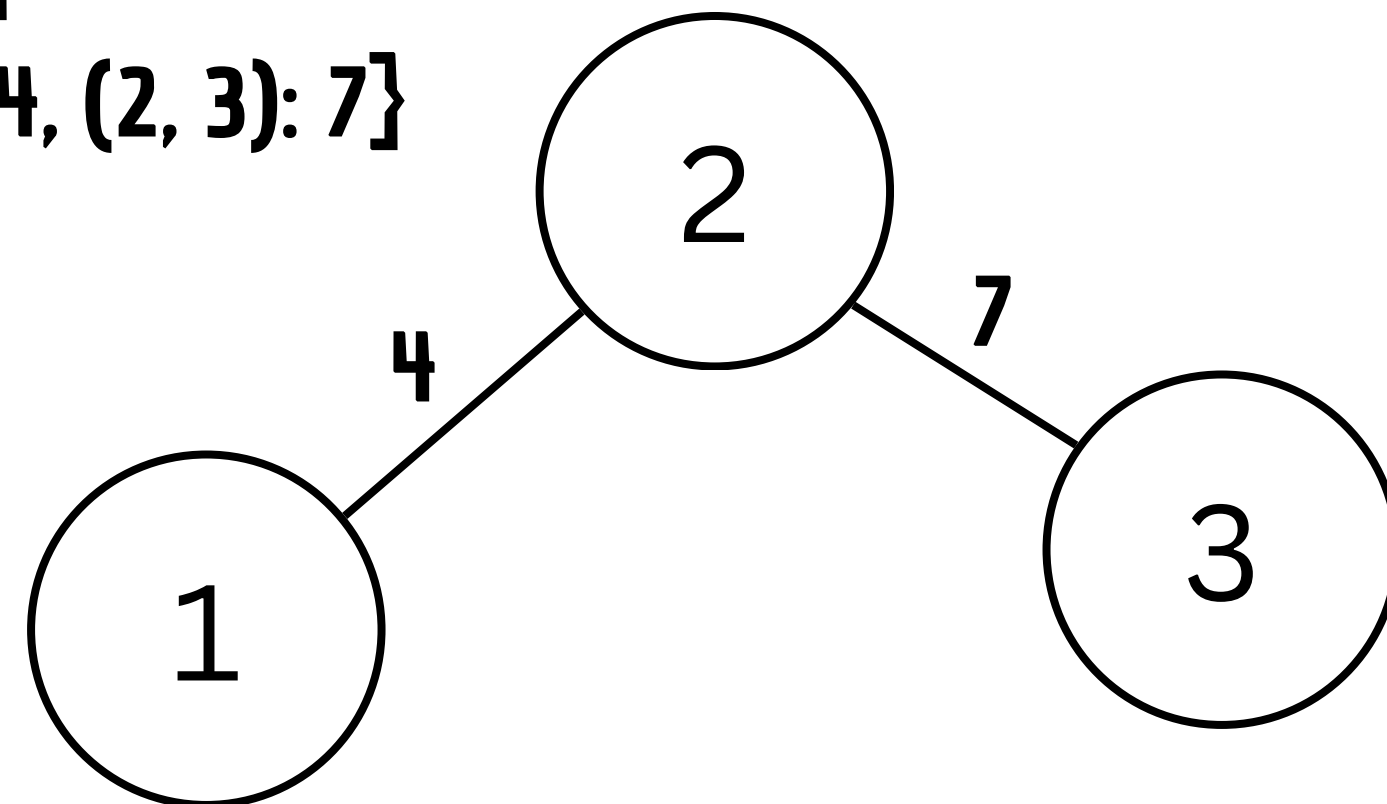
Agreguemos pesos (números) asociados a las aristas:

V = conjunto de nodos

Ej: $\{1, 2, 3\}$

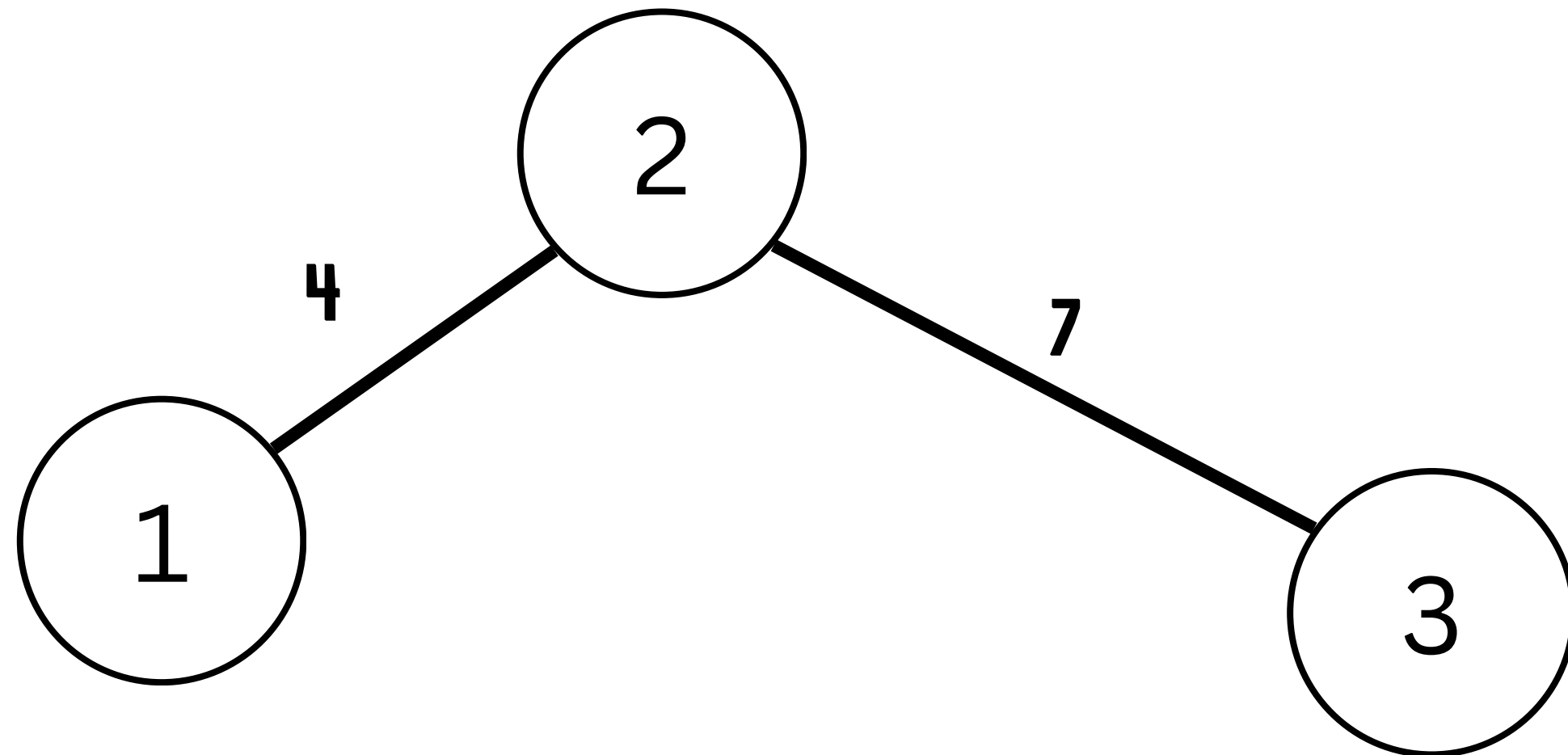
E = conjunto de aristas

Ej: $\{(1, 2): 4, (2, 3): 7\}$



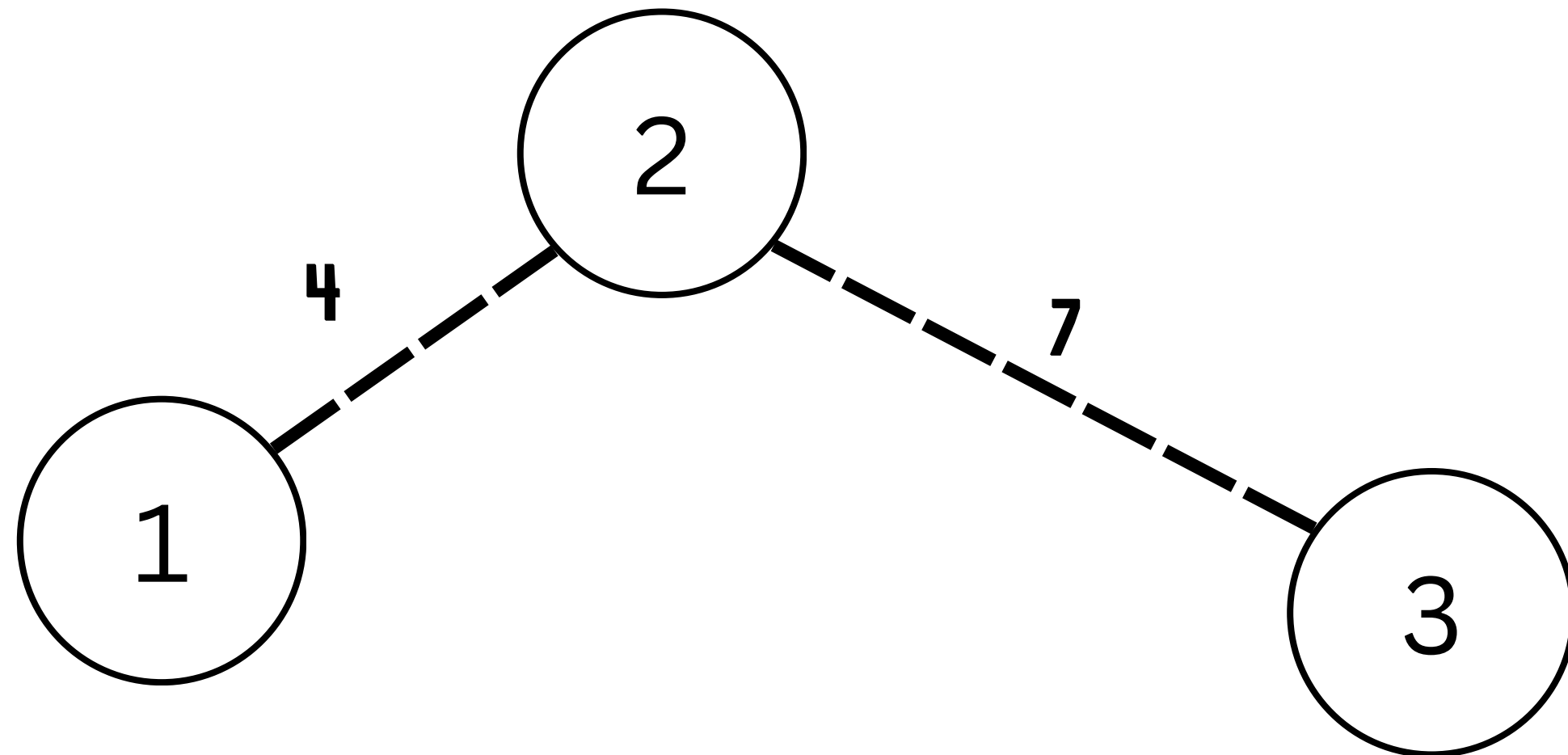
ARISTAS CON “LARGO”

Podemos interpretar los pesos de las aristas como el largo de la arista



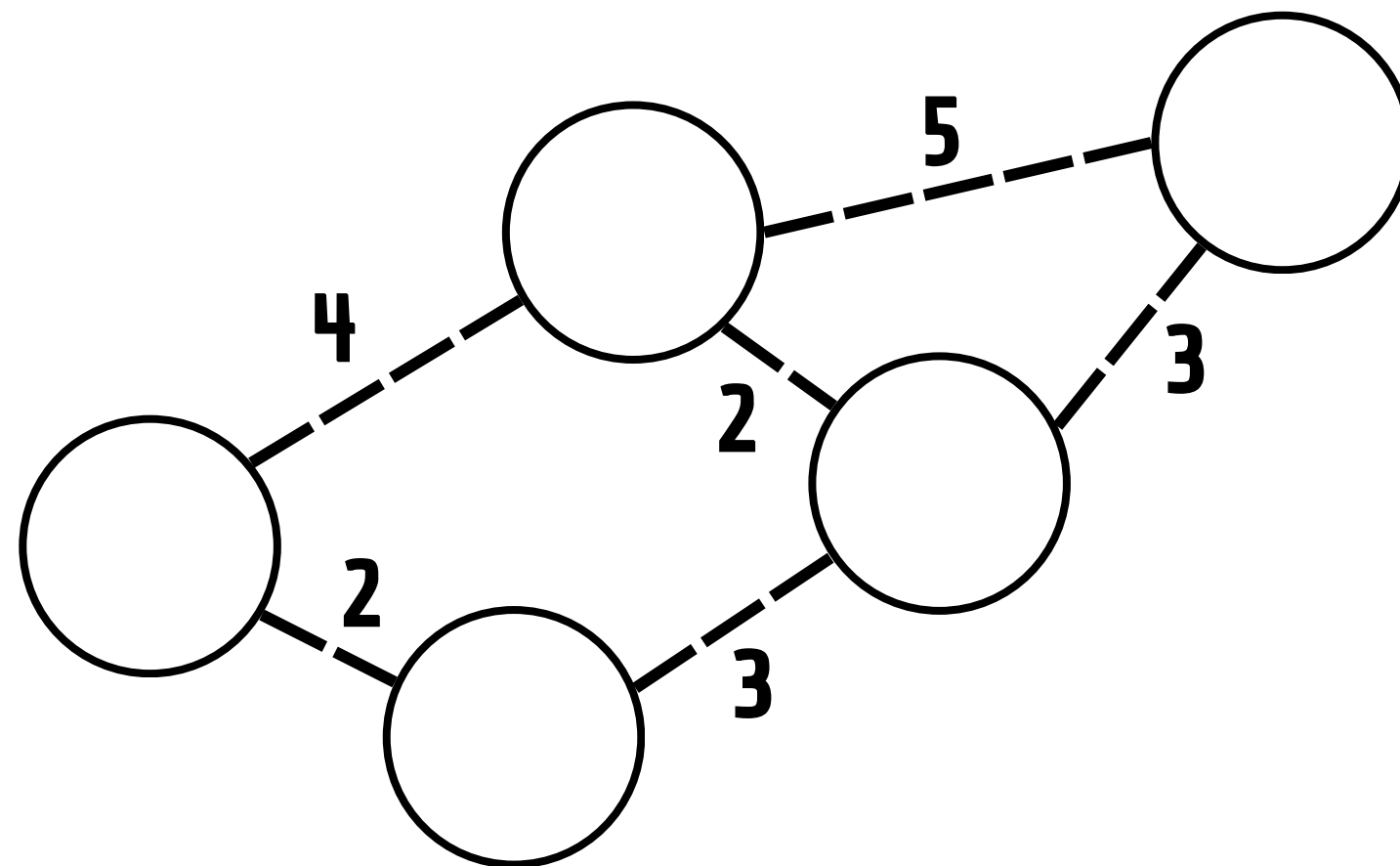
ARISTAS CON “LARGO”

Podemos interpretar los pesos de las aristas como el largo de la arista



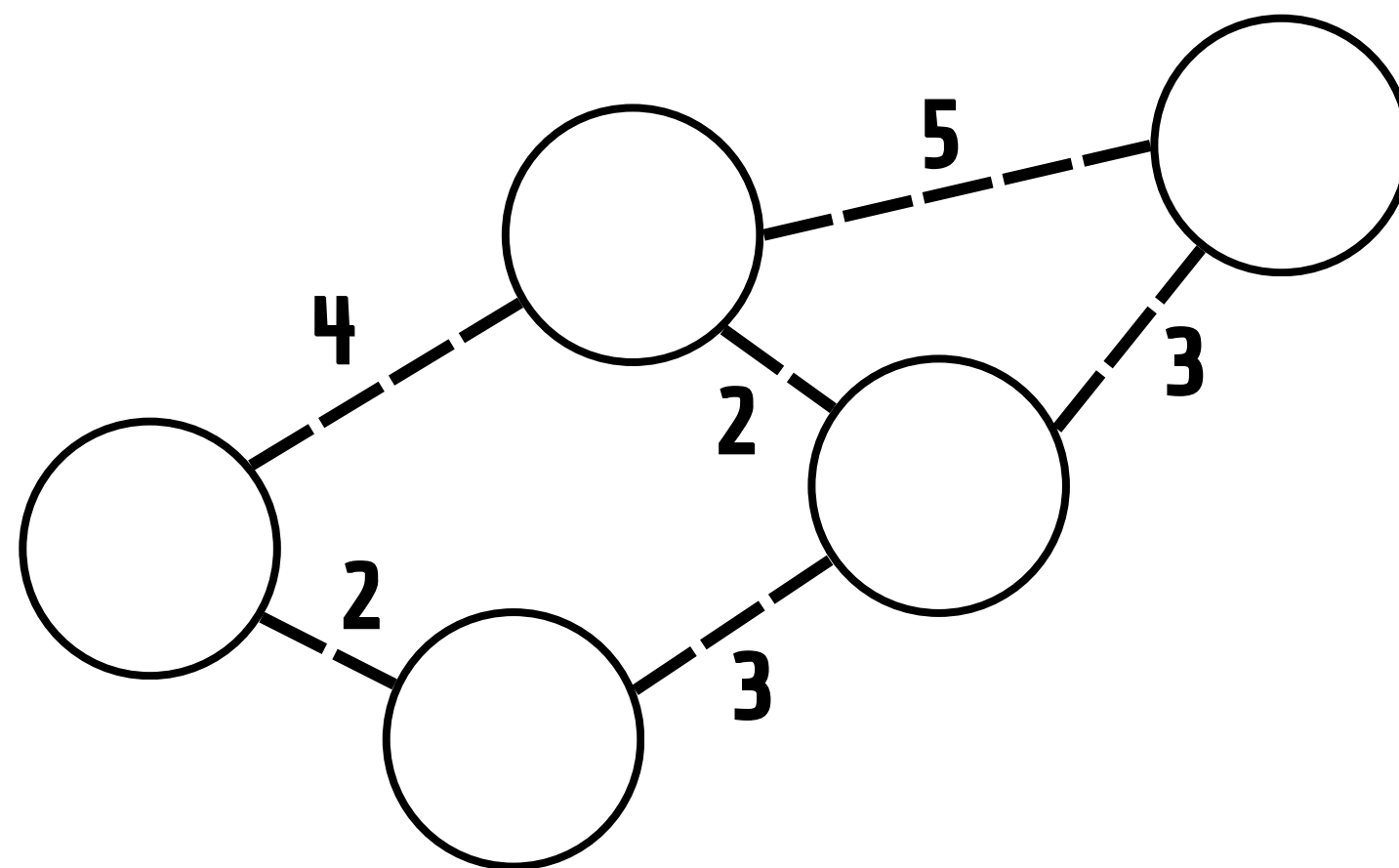
SINGLE-SOURCE-SHORTEST-PATH

Resolvamos un problema clásico: las distancias desde un nodo a todo el resto.



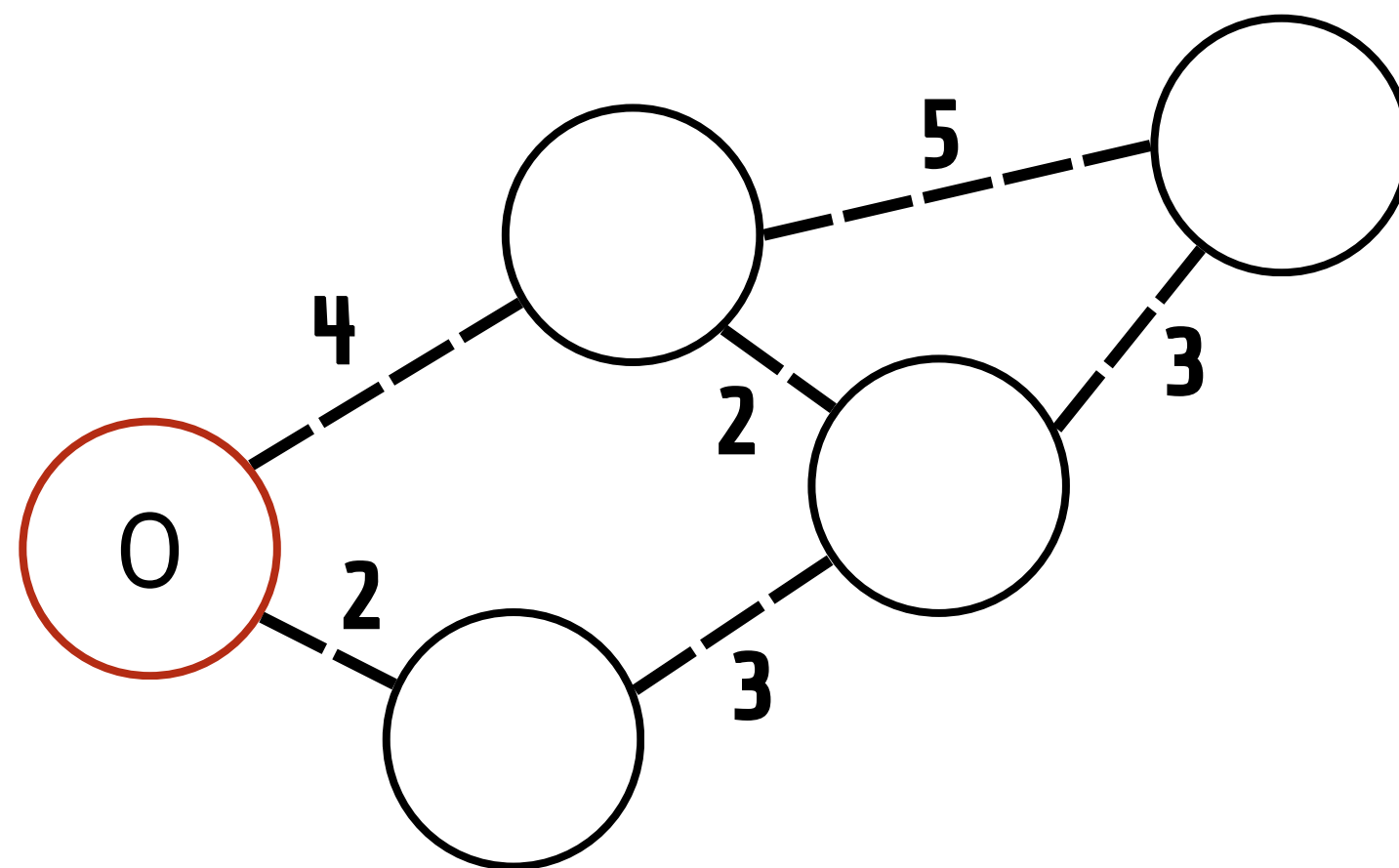
BFS: ARISTAS CON LARGO

Conocemos una herramienta: BFS. ¿Cómo lo hacemos cuando las aristas tienen largo?



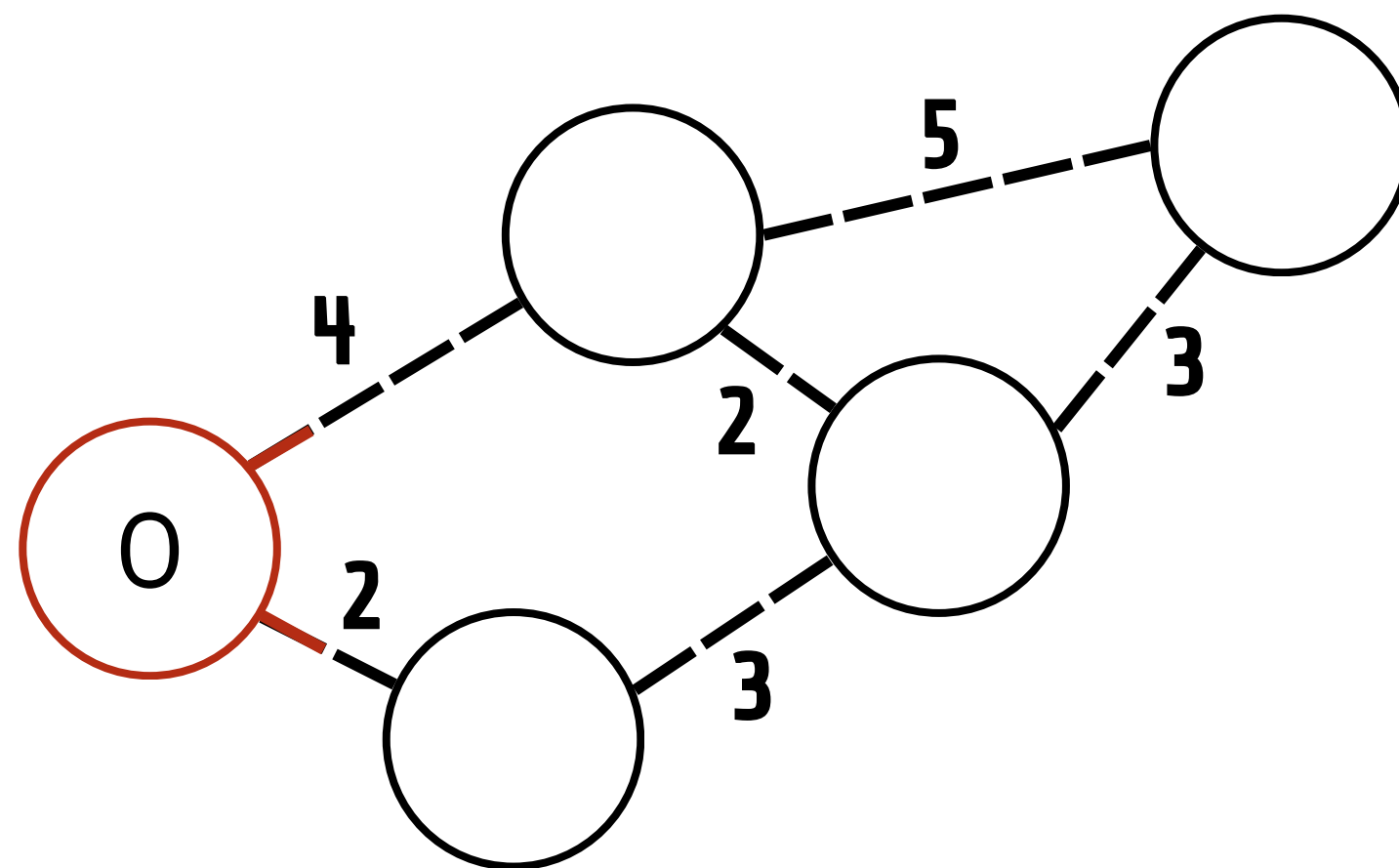
BFS: ARISTAS CON LARGO

Conocemos una herramienta: BFS. ¿Cómo lo hacemos cuando las aristas tienen largo?



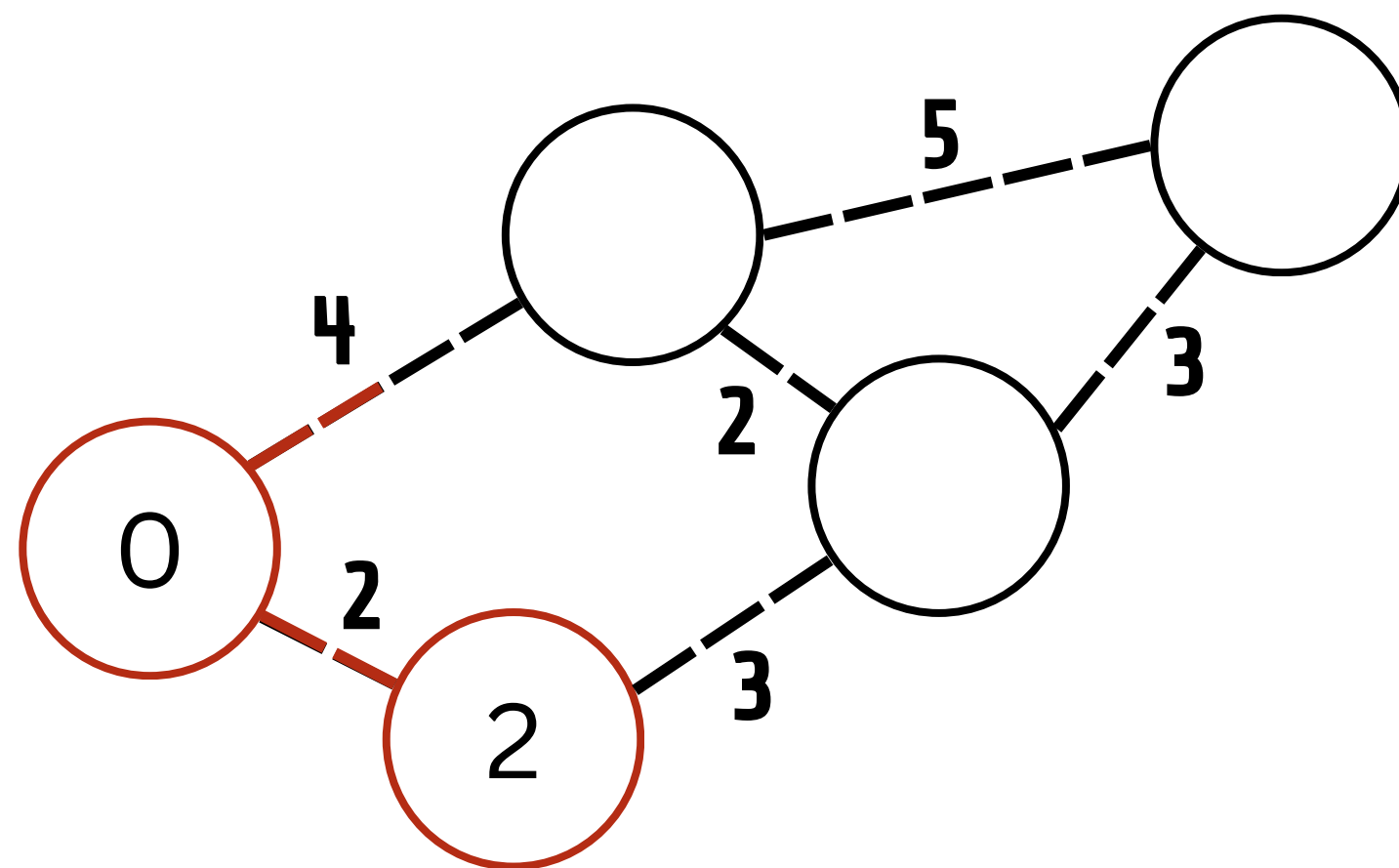
BFS: ARISTAS CON LARGO

Conocemos una herramienta: BFS. ¿Cómo lo hacemos cuando las aristas tienen largo?



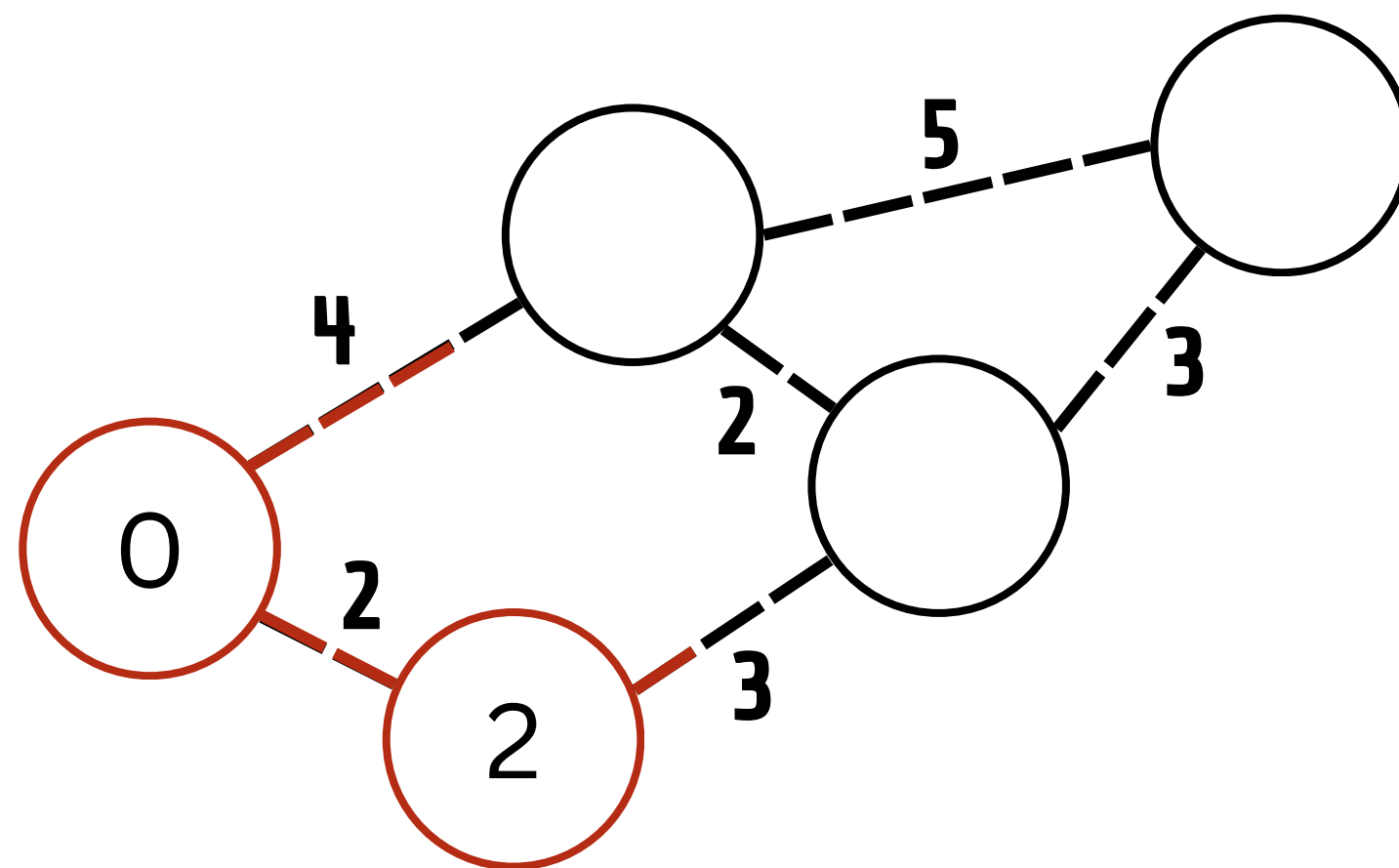
BFS: ARISTAS CON LARGO

Conocemos una herramienta: BFS. ¿Cómo lo hacemos cuando las aristas tienen largo?



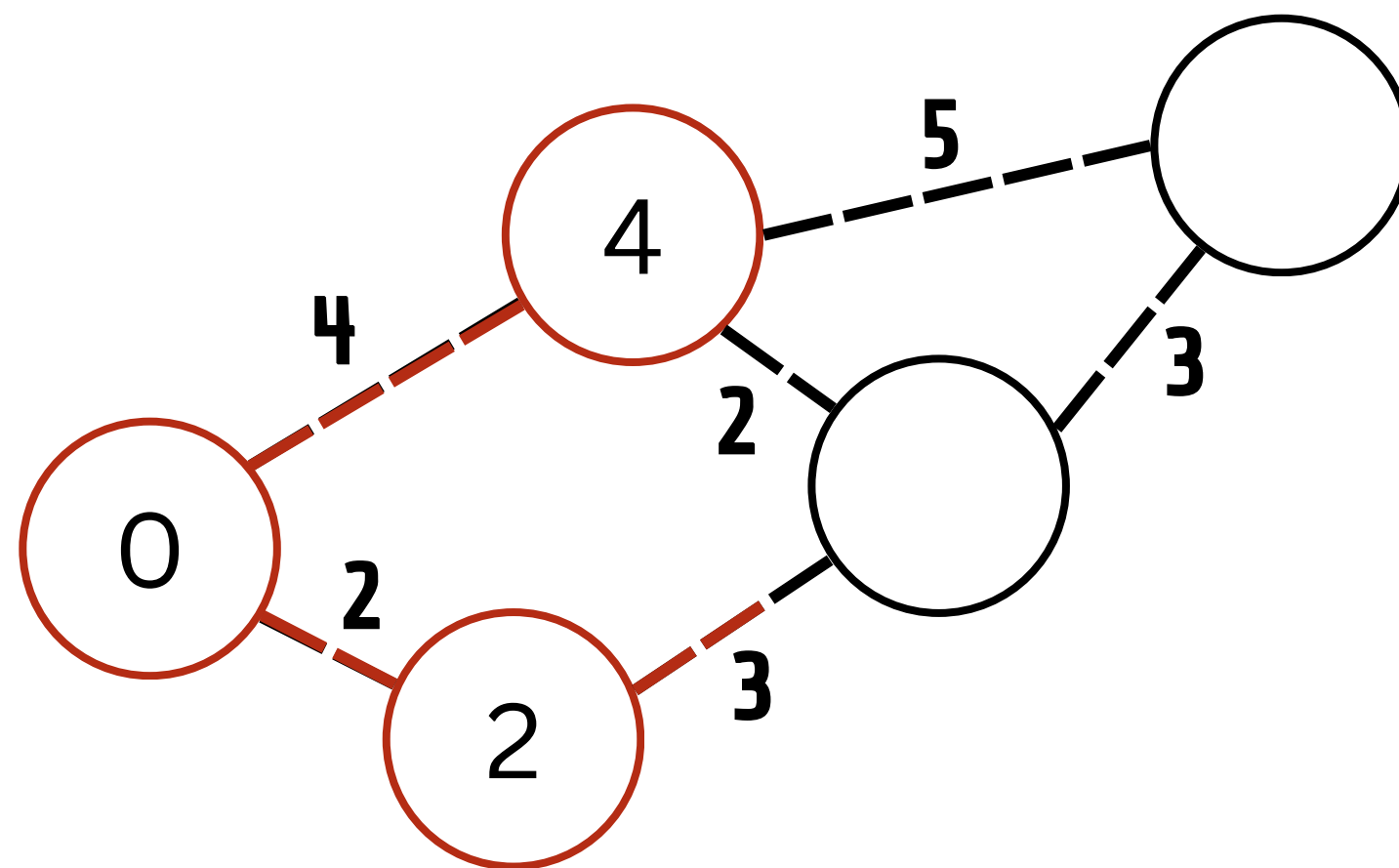
BFS: ARISTAS CON LARGO

Conocemos una herramienta: BFS. ¿Cómo lo hacemos cuando las aristas tienen largo?



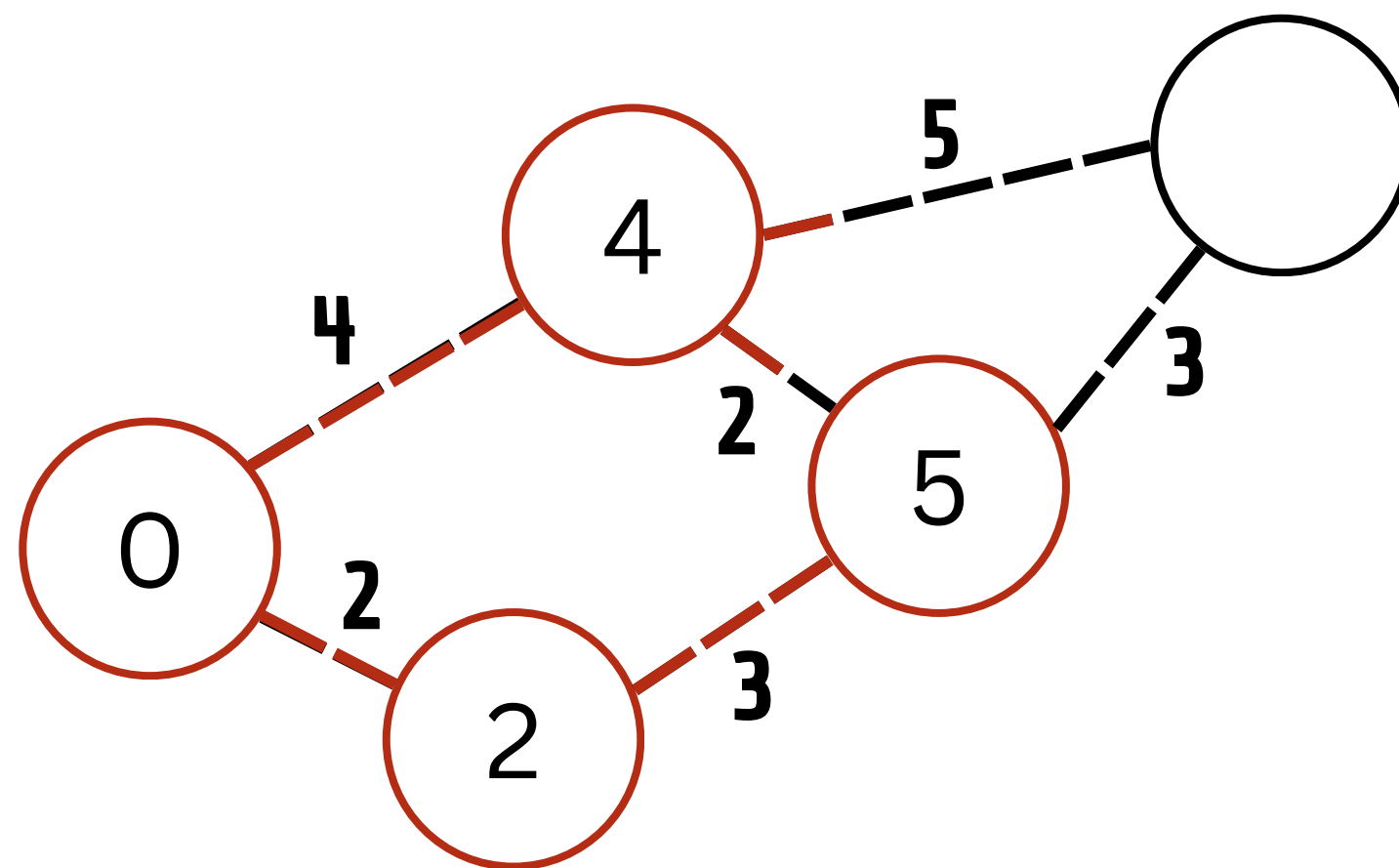
BFS: ARISTAS CON LARGO

Conocemos una herramienta: BFS. ¿Cómo lo hacemos cuando las aristas tienen largo?



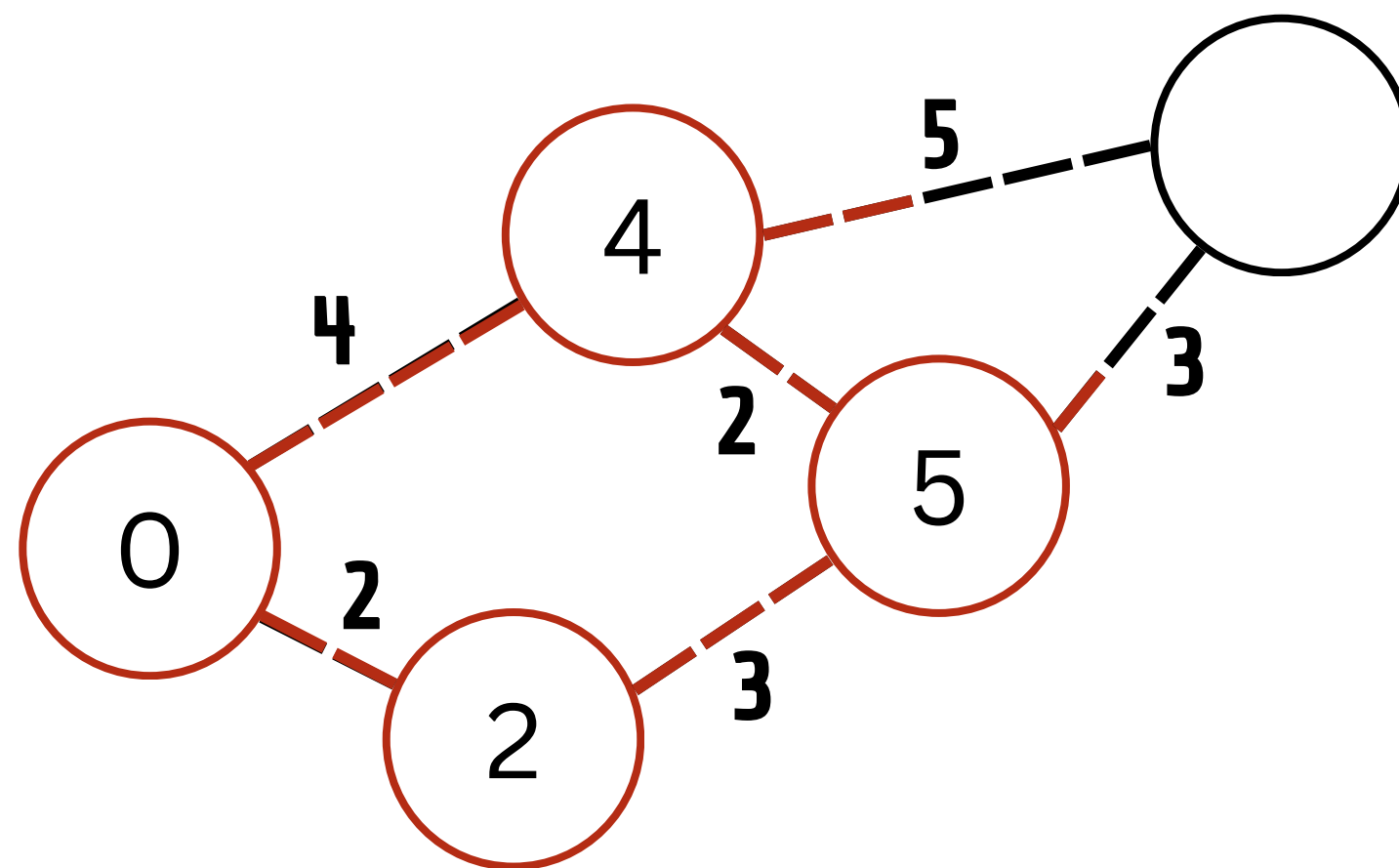
BFS: ARISTAS CON LARGO

Conocemos una herramienta: BFS. ¿Cómo lo hacemos cuando las aristas tienen largo?



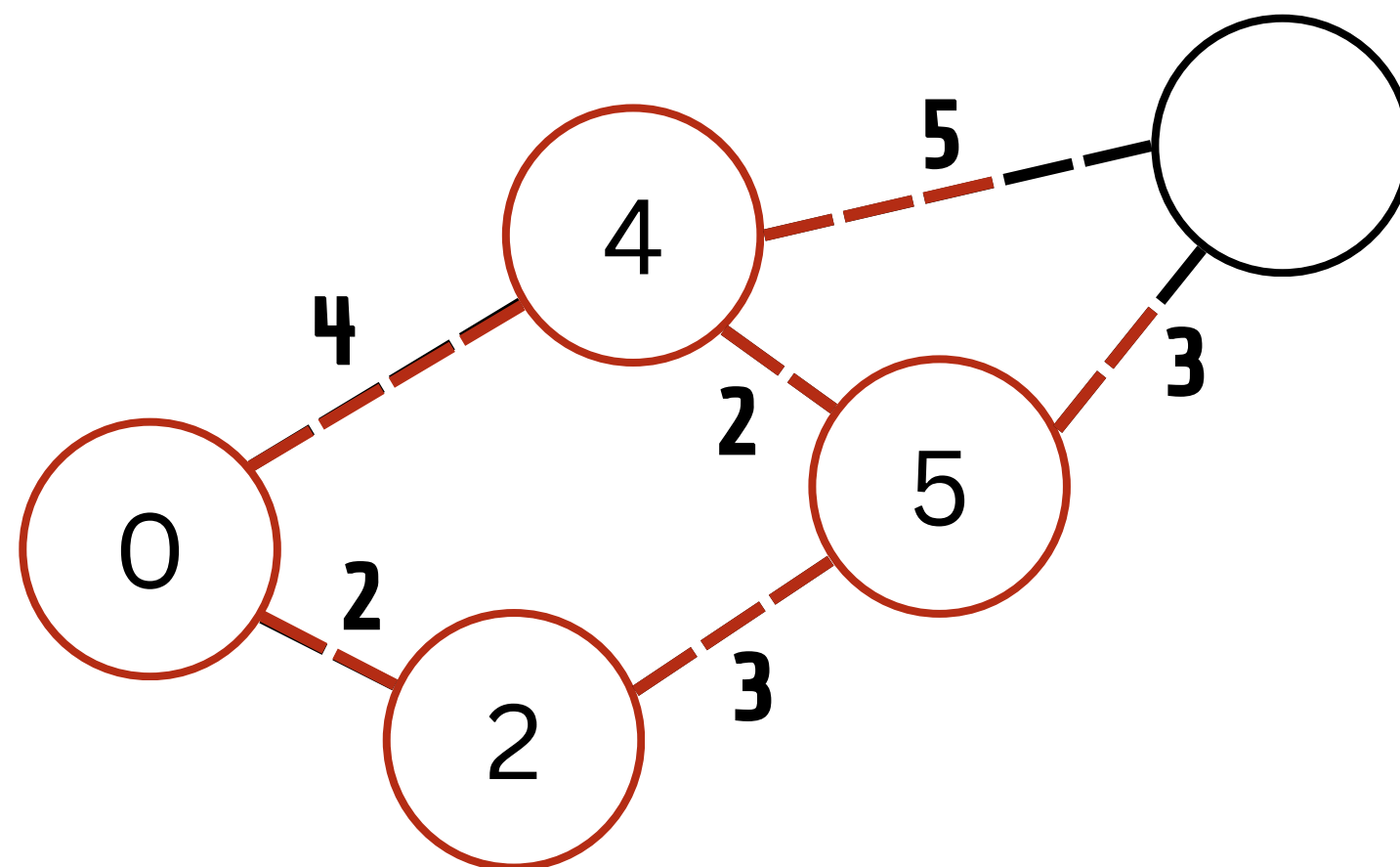
BFS: ARISTAS CON LARGO

Conocemos una herramienta: BFS. ¿Cómo lo hacemos cuando las aristas tienen largo?



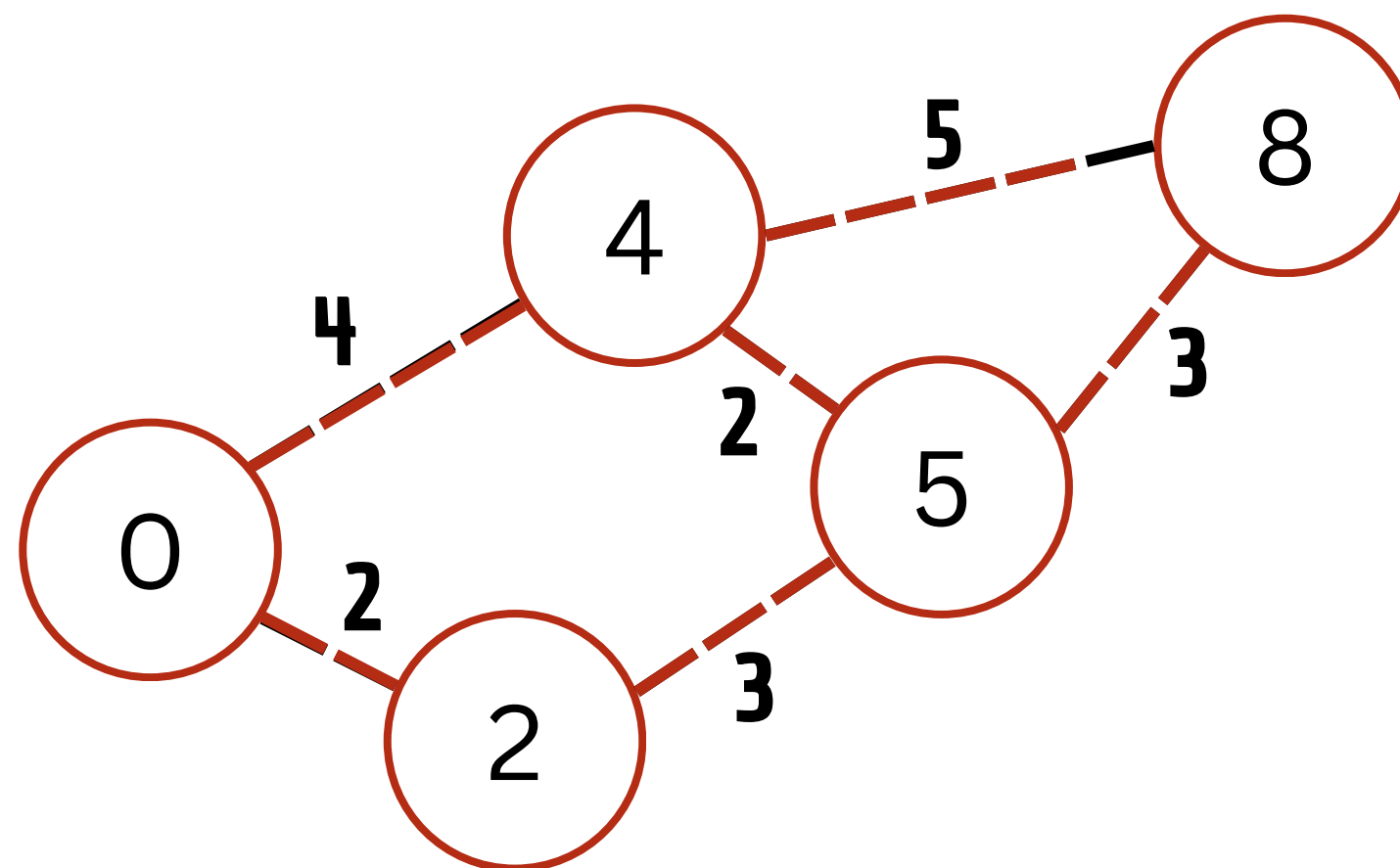
BFS: ARISTAS CON LARGO

Conocemos una herramienta: BFS. ¿Cómo lo hacemos cuando las aristas tienen largo?



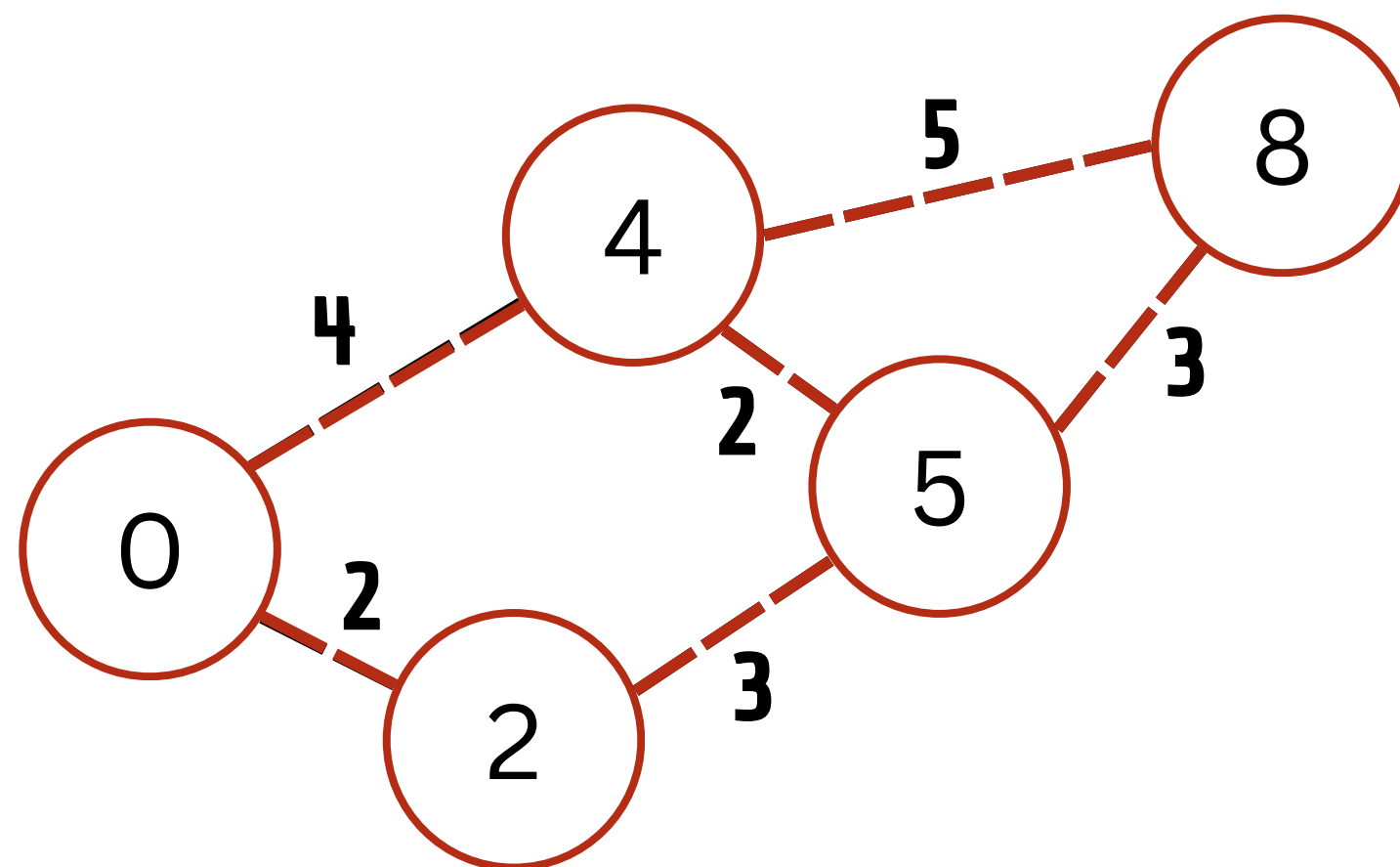
BFS: ARISTAS CON LARGO

Conocemos una herramienta: BFS. ¿Cómo lo hacemos cuando las aristas tienen largo?



BFS: ARISTAS CON LARGO

Conocemos una herramienta: BFS. ¿Cómo lo hacemos cuando las aristas tienen largo?



COMPLEJIDAD DE BFS

Complejidad = Cantidad de pasos que hicimos

Complejidad: $O\left(\sum_{e \in E} e_w\right)$

COMPLEJIDAD DE BFS

Complejidad = Cantidad de pasos que hicimos

Complejidad: $O\left(\sum_{e \in E} e_w\right)$

¡Podemos hacerlo mejor!

DIJKSTRA

El algoritmo es simplemente una optimización del BFS: Avancemos todo lo que se pueda de a una.

DIJKSTRA: PSEUDOCÓDIGO

$q \leftarrow \{0\}$

$D \leftarrow [\text{INF for } _ \text{ in range}(n)]$

while not $q.empty()$:

$u \leftarrow (\text{sacar de } q \text{ un elemento con mínimo } D[u])$

 for (v, w) in (aristas adyacentes a u):

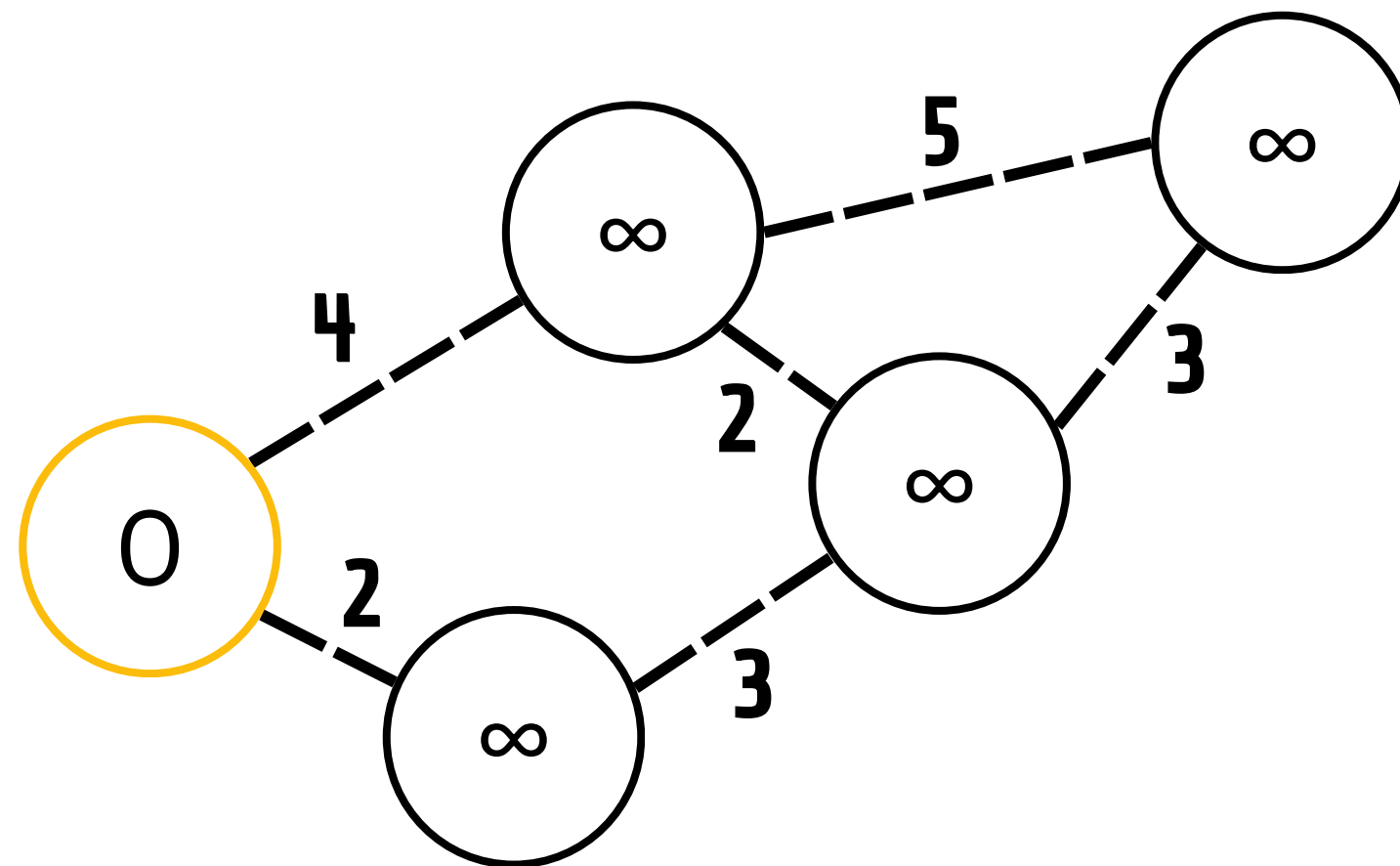
 if $D[u] + w < D[v]$:

$D[v] \leftarrow D[u] + w$

 (agregar v a q)

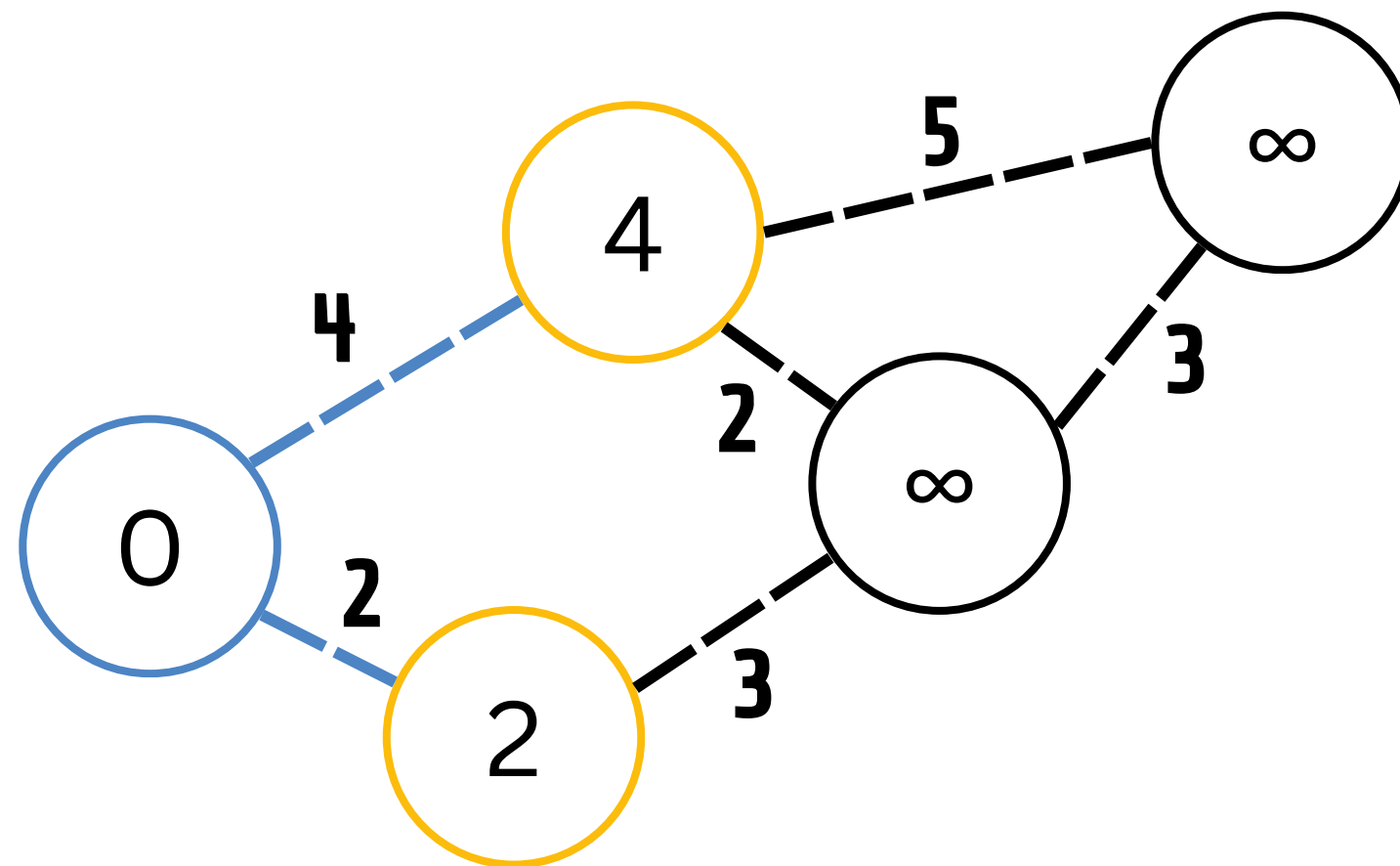
DIJKSTRA

nodos en q = amarillo **nodos aún no vistos = negro** **nodos sacados de q = rojo**



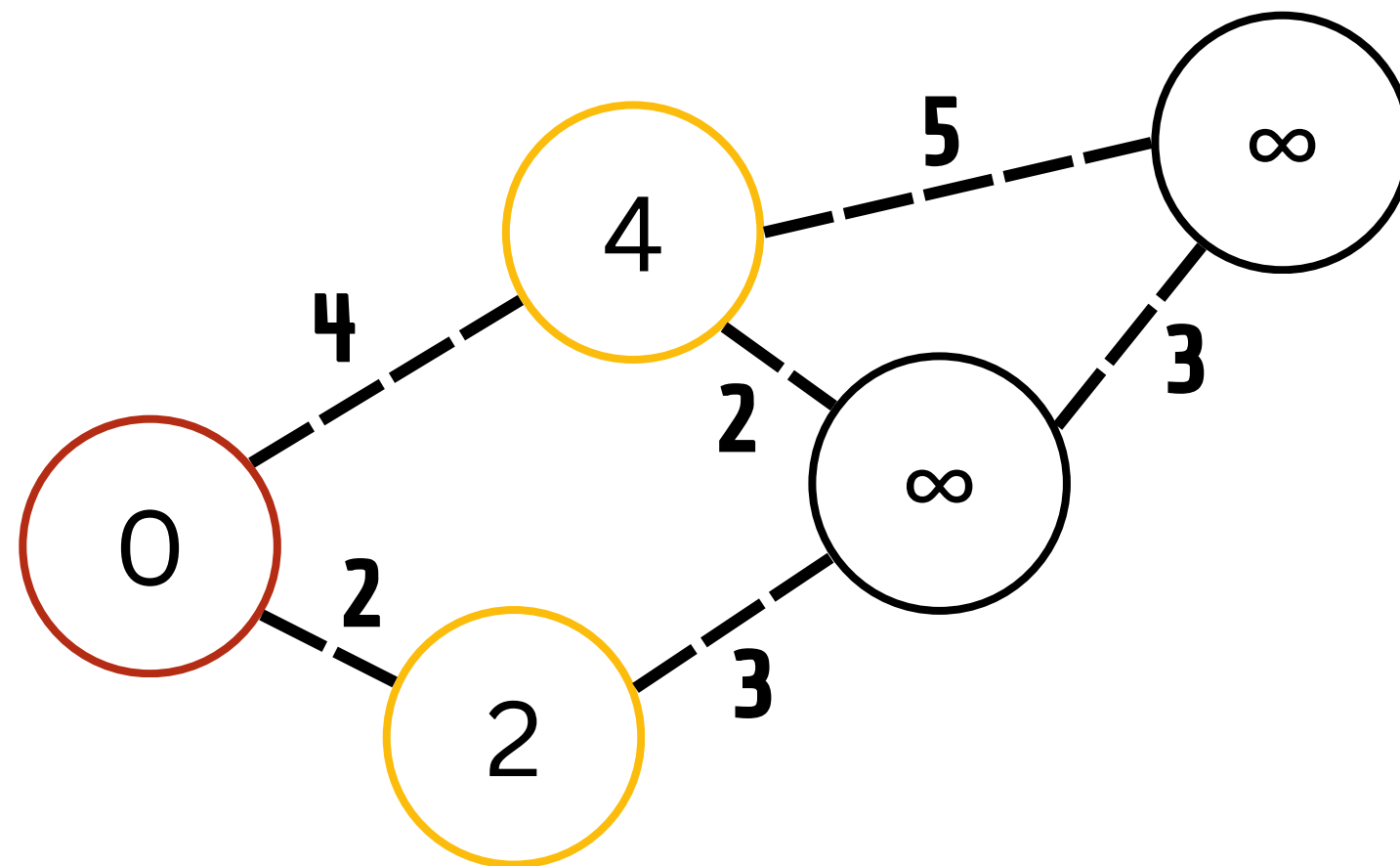
DIJKSTRA

nodos en q = amarillo **nodos aún no vistos = negro** **nodos sacados de q = rojo**



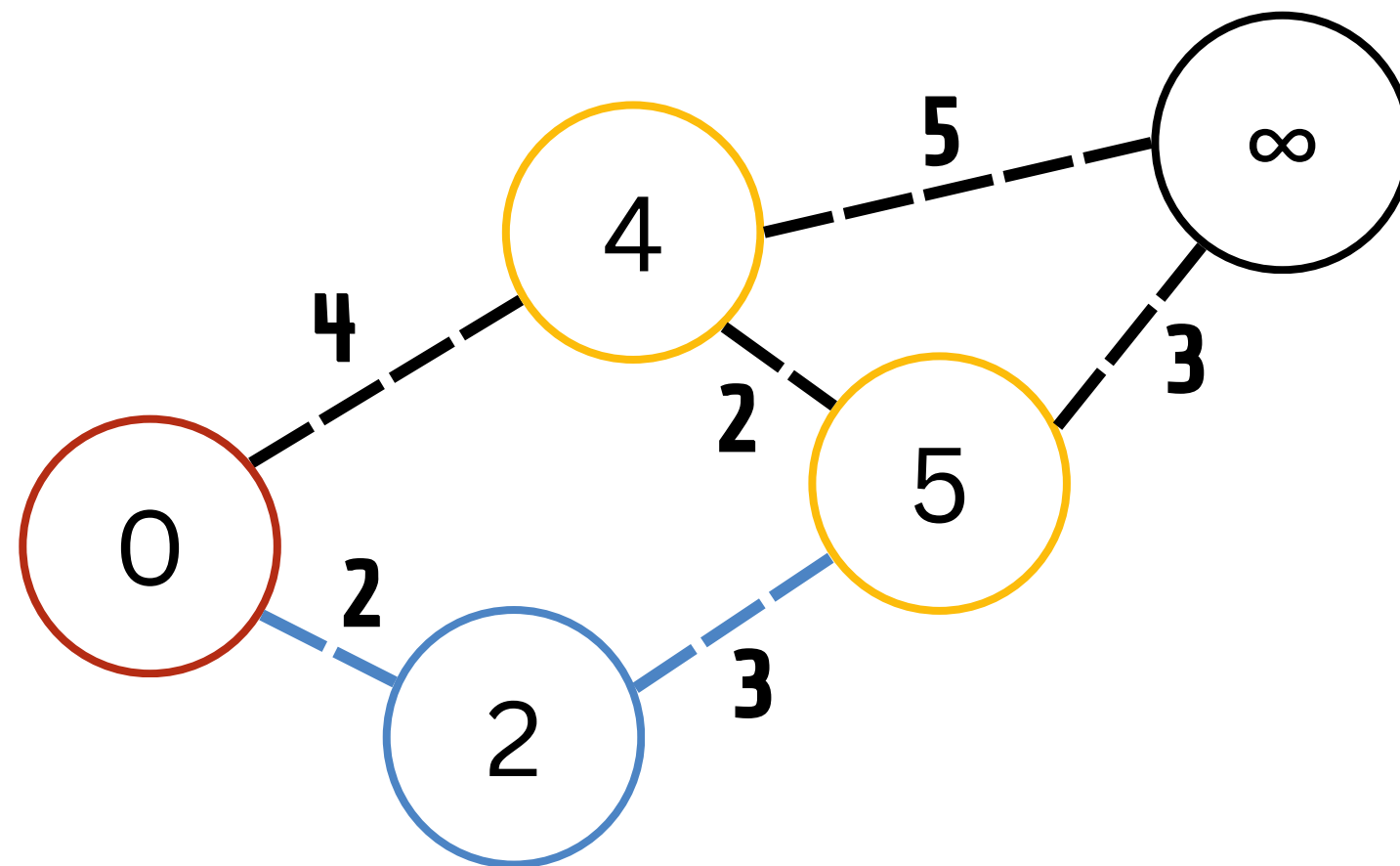
DIJKSTRA

nodos en q = amarillo **nodos aún no vistos = negro** **nodos sacados de q = rojo**



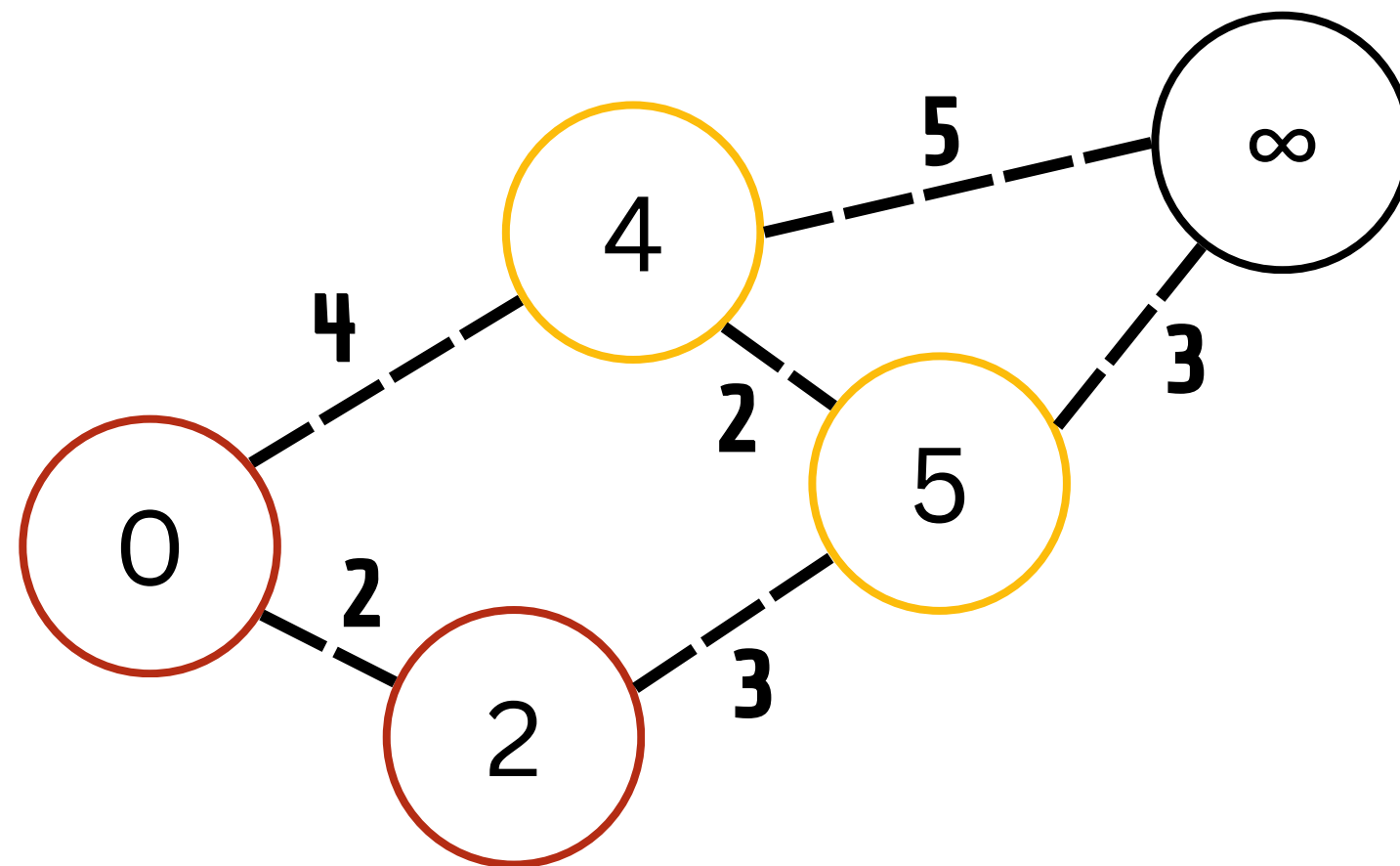
DIJKSTRA

nodos en q = amarillo **nodos aún no vistos = negro** **nodos sacados de q = rojo**



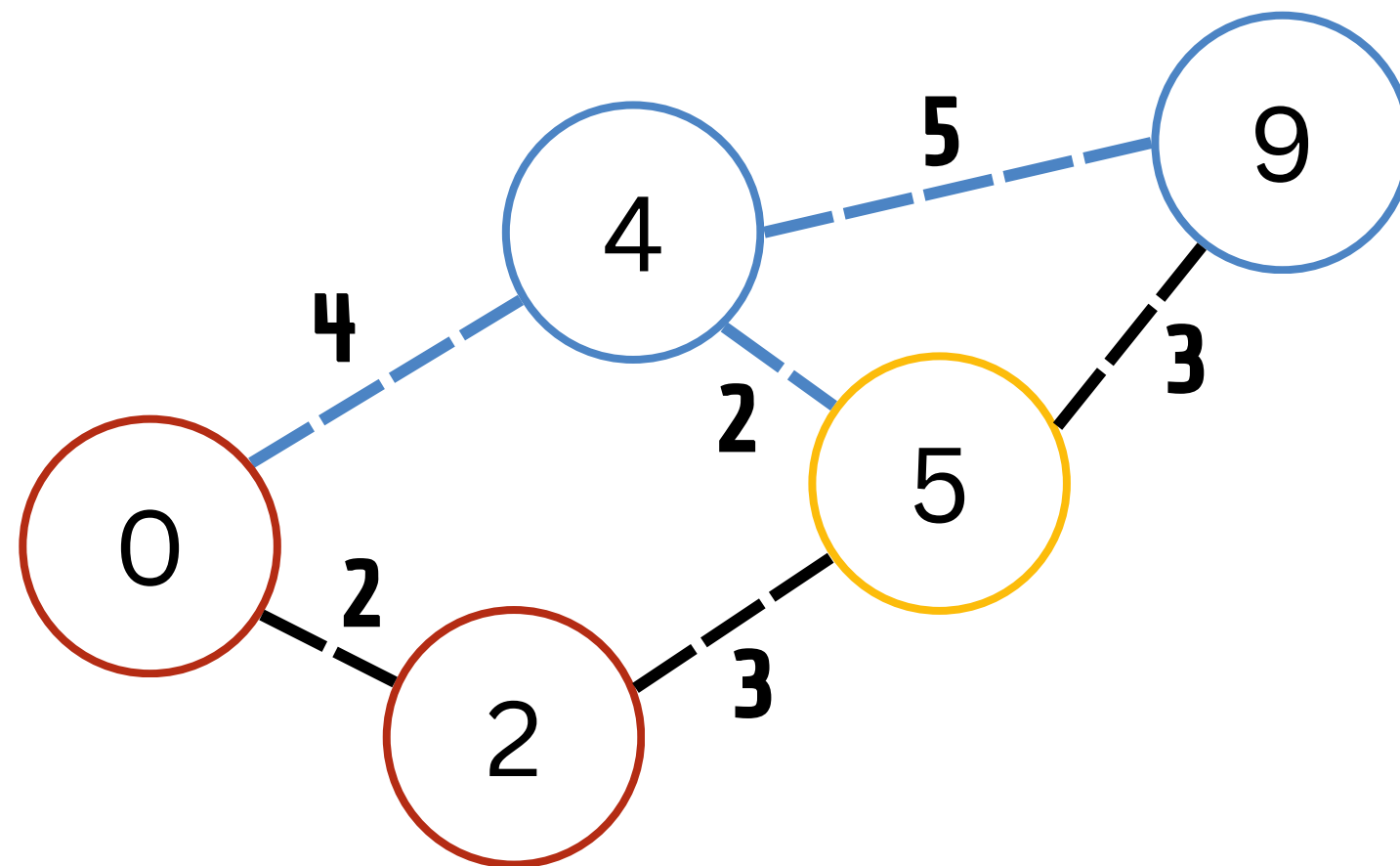
DIJKSTRA

nodos en q = amarillo **nodos aún no vistos = negro** **nodos sacados de q = rojo**



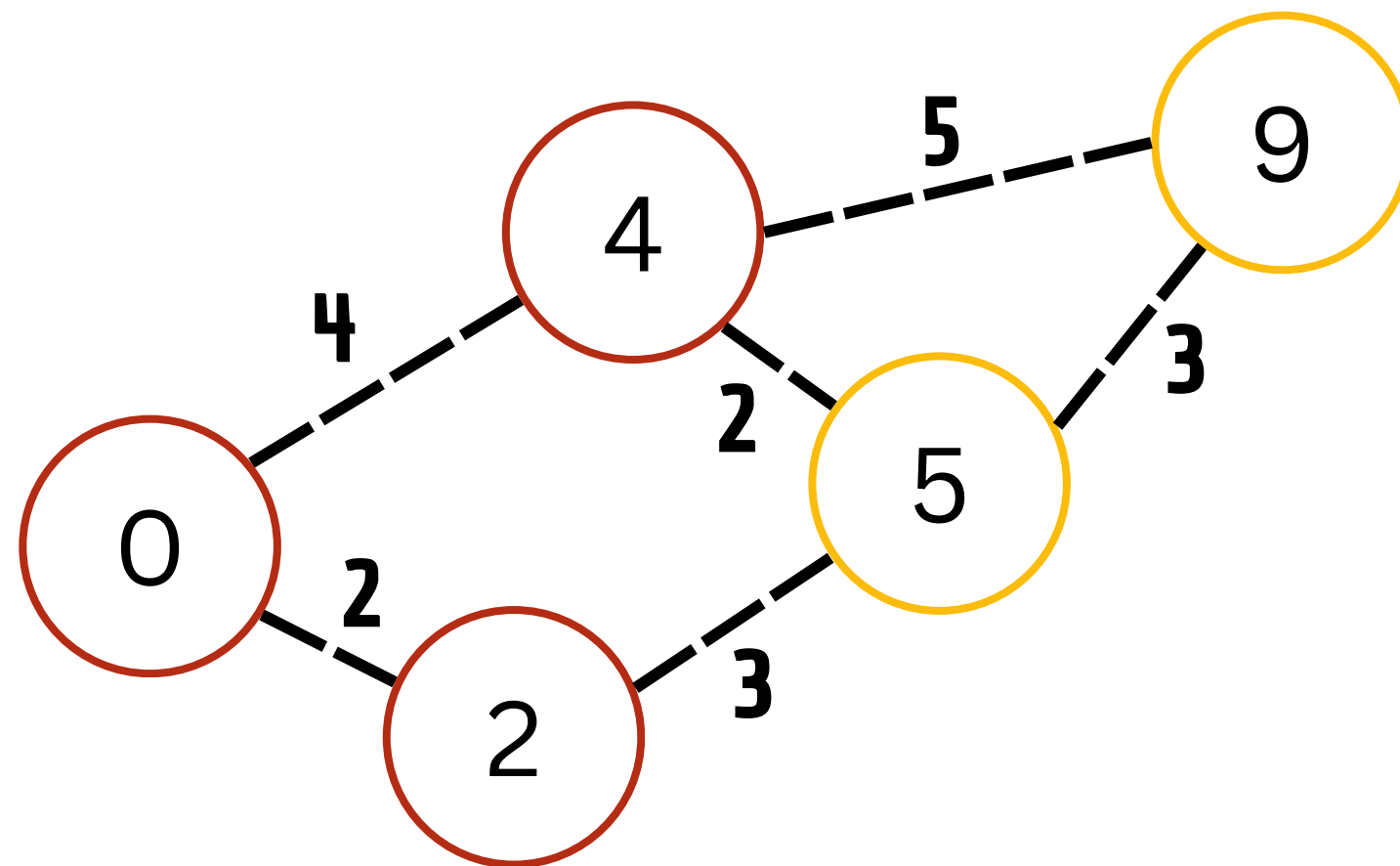
DIJKSTRA

nodos en q = amarillo **nodos aún no vistos = negro** **nodos sacados de q = rojo**



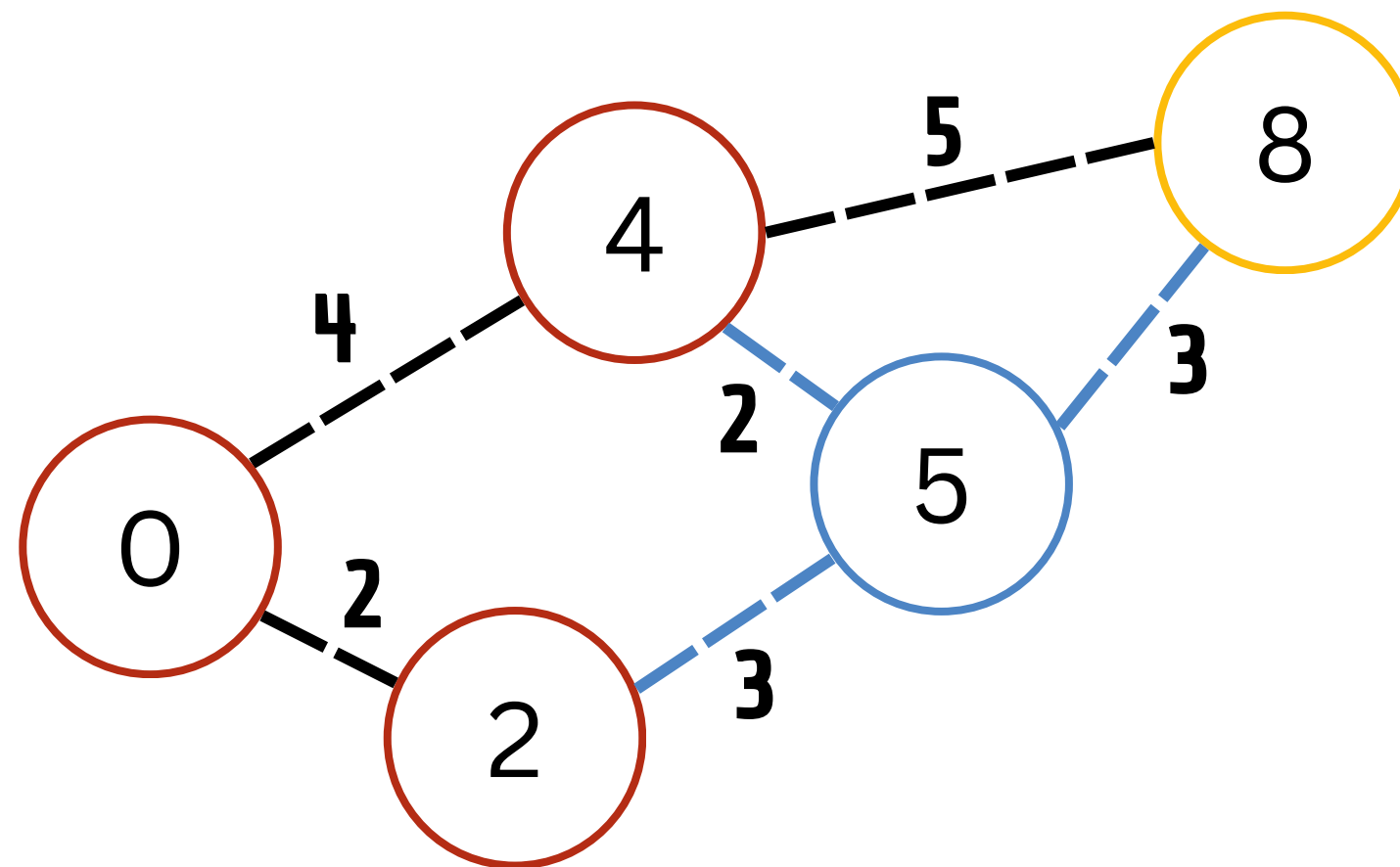
DIJKSTRA

nodos en q = amarillo **nodos aún no vistos = negro** **nodos sacados de q = rojo**



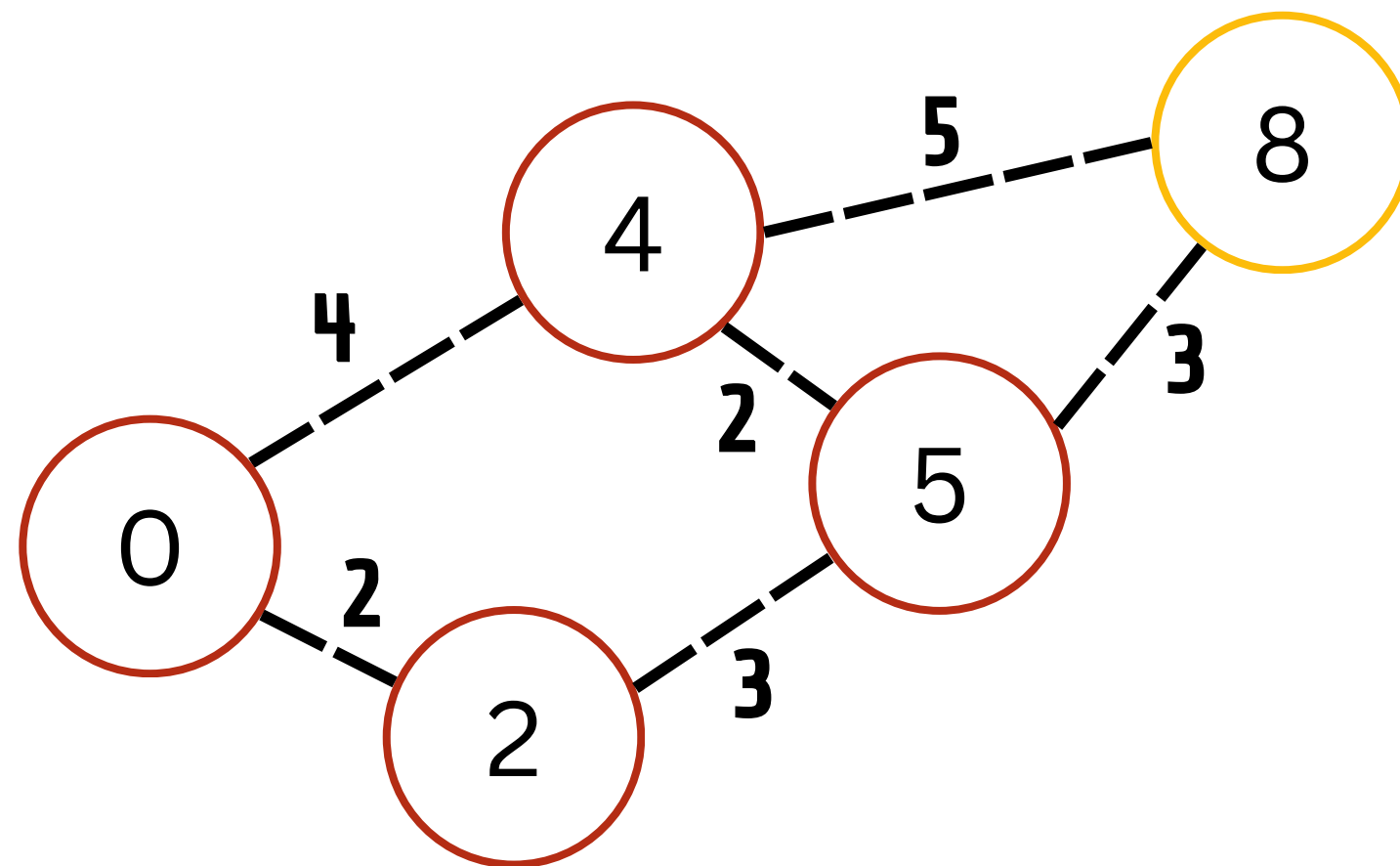
DIJKSTRA

nodos en q = amarillo **nodos aún no vistos = negro** **nodos sacados de q = rojo**



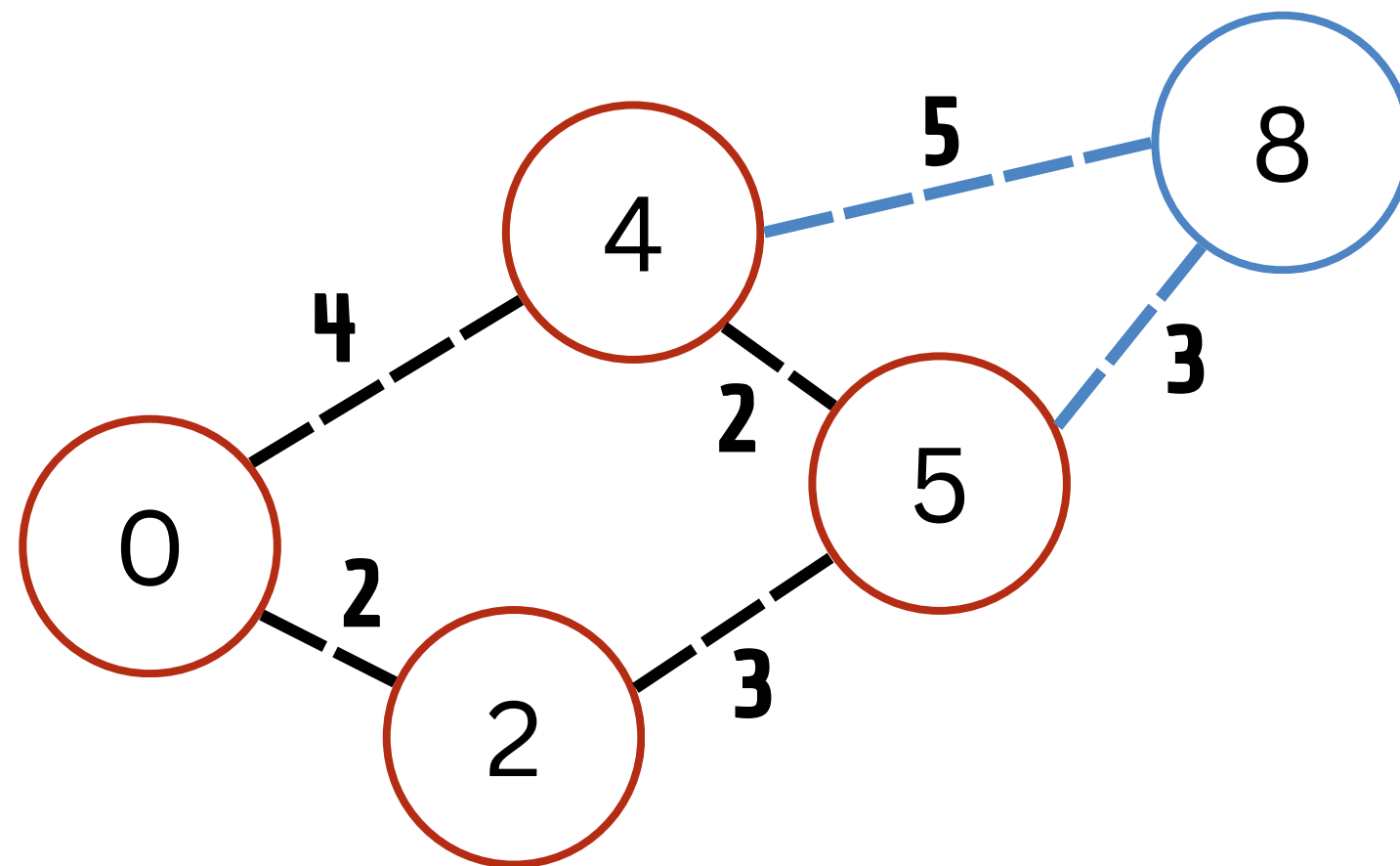
DIJKSTRA

nodos en q = amarillo **nodos aún no vistos = negro** **nodos sacados de q = rojo**



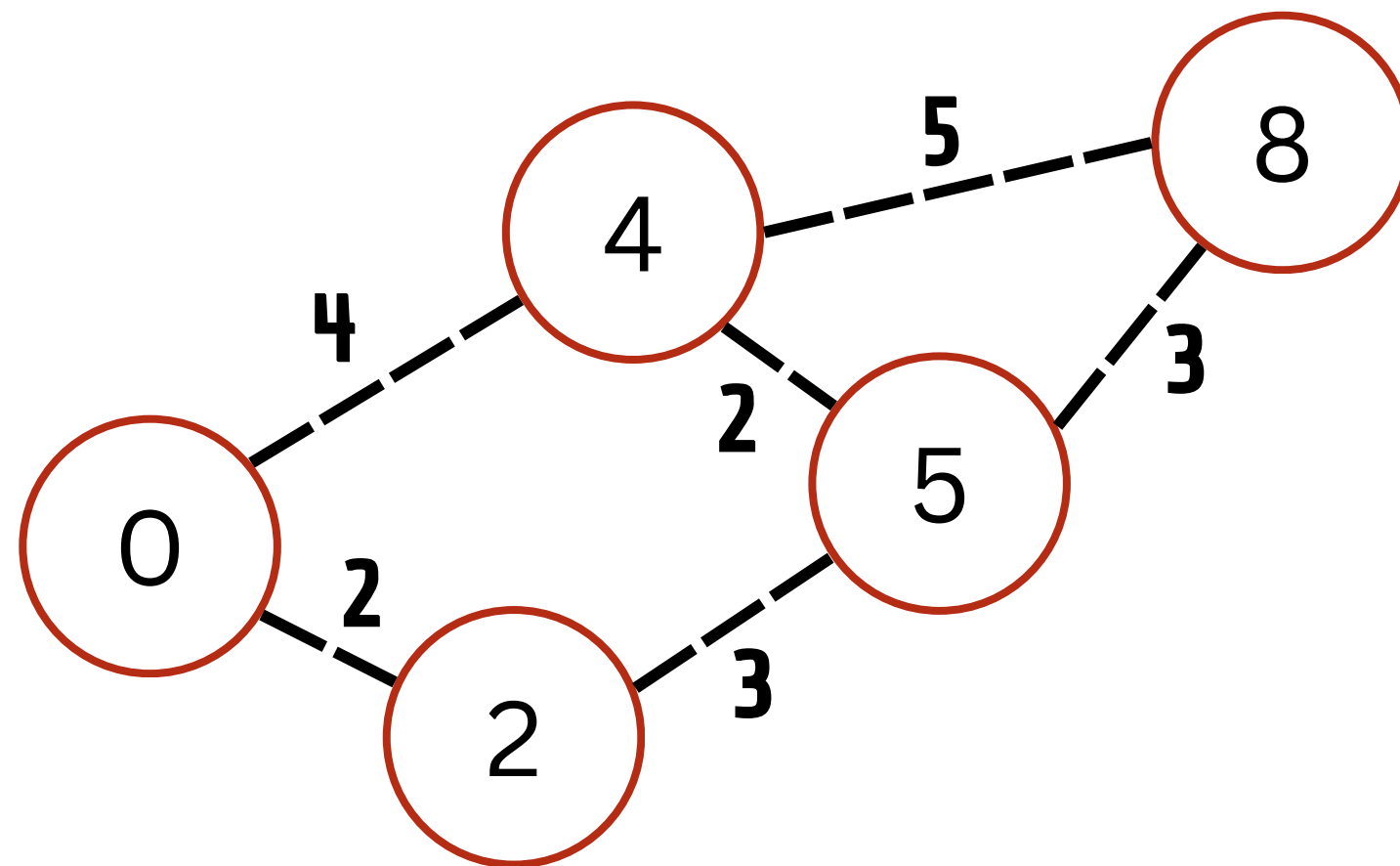
DIJKSTRA

nodos en q = amarillo **nodos aún no vistos = negro** **nodos sacados de q = rojo**



DIJKSTRA

nodos en q = amarillo **nodos aún no vistos = negro** **nodos sacados de q = rojo**



DIJKSTRA: COMPLEJIDAD?

$q \leftarrow \{0\}$

$D \leftarrow [\text{INF for } _ \text{ in range}(n)]$

while not $q.empty()$: $\longleftarrow$ **Una vez por cada nodo**

$u \leftarrow$ (sacar de q un elemento con mínimo $D[u]$)

for (v, w) in (aristas adyacentes a u):

if $D[u] + w < D[v]$: $\nwarrow$ **Para cada nodo, sus aristas.**

$D[v] \leftarrow D[u] + w$ **Total para todos los nodos: $|E|$**

(agregar v a q)

DIJKSTRA: COMPLEJIDAD?

```
q ← {0}
D ← [INF for _ in range(n)]
while not q.empty():
    u ← (sacar de q un elemento con mínimo D[u])
    for (v, w) in (aristas adyacentes a u):
        if D[u] + w < D[v]:
            D[v] ← D[u] + w
            (agregar v a q)
```

PRIORITY QUEUE AL RESCATE

Justamente tenemos una estructura de datos que nos permite mantener un conjunto y obtener el más pequeño: las colas de prioridad.

Operación *insert*: $O(\log N)$

Operación *pop-min*: $O(\log N)$

DIJKSTRA: CASI-CÓDIGO

```
q ← priority_queue(); q.insert( (0, 0) )
D ← [INF for _ in range(n)]
while not q.empty():
    d, u ← q.pop()
    if d > D[u]: continue
    for (v, w) in (aristas adyacentes a u):
        if D[u] + w < D[v]:
            D[v] ← D[u] + w
            q.push((D[v], v))
```

DIJKSTRA: COMPLEJIDAD

```
q ← priority_queue(); q.insert( (0, 0) )
D ← [INF for _ in range(n)] ← |V|
while not q.empty():
    d, u ← q.pop()
    if d > D[u]: continue
    for (v, w) in (aristas adyacentes a u):
        if D[u] + w < D[v]:
            D[v] ← D[u] + w
            q.push((D[v], v)) ← |E| * log |q| ≤ |E| * log |E|
```

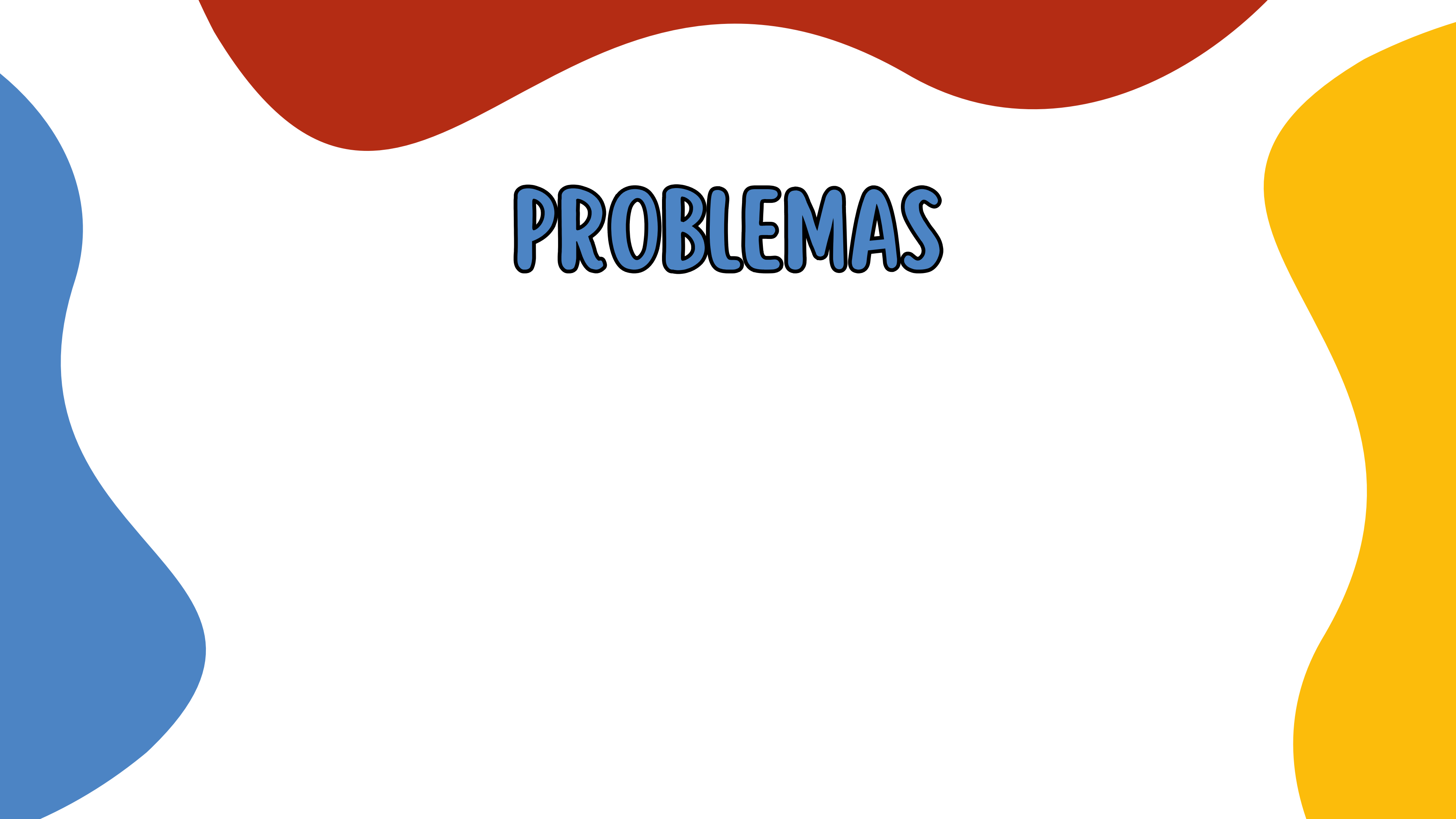
DIJKSTRA: COMPLEJIDAD

Podemos calcular la complejidad de Dijkstra:

$$O(|V| + |E| \log |E|)$$

NOTA:

$$\log |E| \leq \log (|V|^2) \leq 2 \log |V|$$
$$O(|V| + |E| \log |E|) = O(|V| + |E| \log |V|)$$



PROBLEMAS

PROBLEMA: ARISTAS DIRIGIDAS

¿Qué pasa si las aristas sólo nos permiten avanzar en una dirección?

Resolvamos el problema de single-source-shortest-path para este caso, utilizando Dijkstra.

PROBLEMA: ARISTAS DIRIGIDAS

¿Qué pasa si las aristas sólo nos permiten avanzar en una dirección?

¡No pasa nada! Con leves modificaciones podemos reutilizar Dijkstra. Si hacemos memoria, no hemos hecho ningún supuesto que nos impida reemplazar las aristas bidireccionales por aristas unidireccionales.

DIJKSTRA: PSEUDOCÓDIGO

$q \leftarrow \{0\}$

$D \leftarrow [\text{INF for } _ \text{ in range}(n)]$

while not $q.empty()$:

$u \leftarrow (\text{sacar de } q \text{ un elemento con mínimo } D[u])$

 for (v, w) in (aristas adyacentes a u):

 if $D[u] + w < D[v]$:

$D[v] \leftarrow D[u] + w$

 (agregar v a q)

DIJKSTRA DIRIGIDO

```
q ← {0}
D ← [INF for _ in range(n)]
while not q.empty():
    u ← (sacar de q un elemento con mínimo D[u])
    for (v, w) in (aristas desde u):
        if D[u] + w < D[v]:
            D[v] ← D[u] + w
            (agregar v a q)
```

PROBLEMA: PESO EN LOS NODOS

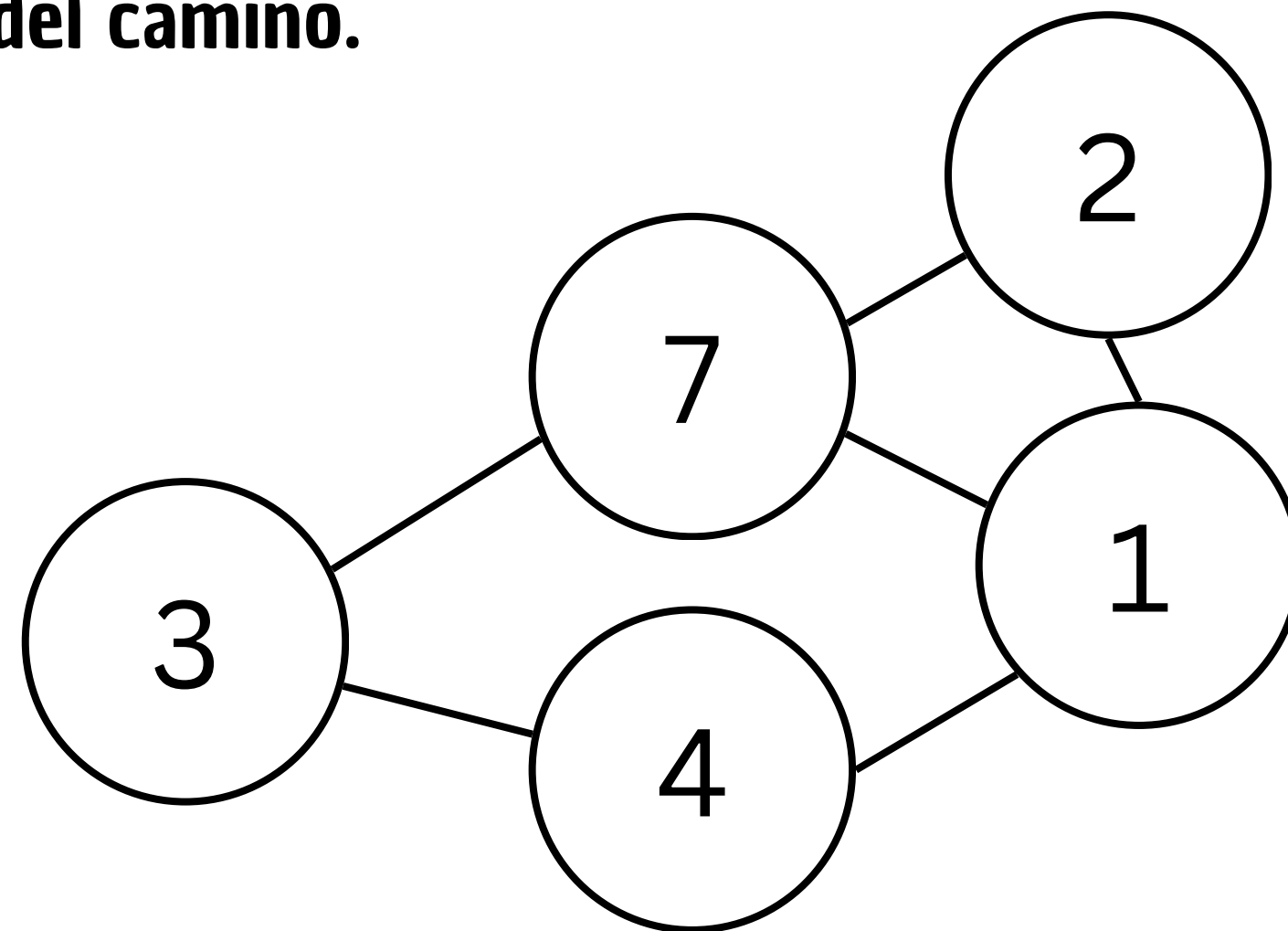
Single-source-shortest-path: Para todo nodo v encontremos la distancia más corta entre 1 y v . Pero usando una definición de distancia alternativa.

Definición original: Suma de las aristas del camino con menor distancia

Nueva definición: Suma de los pesos de los nodos del camino con menor distancia.

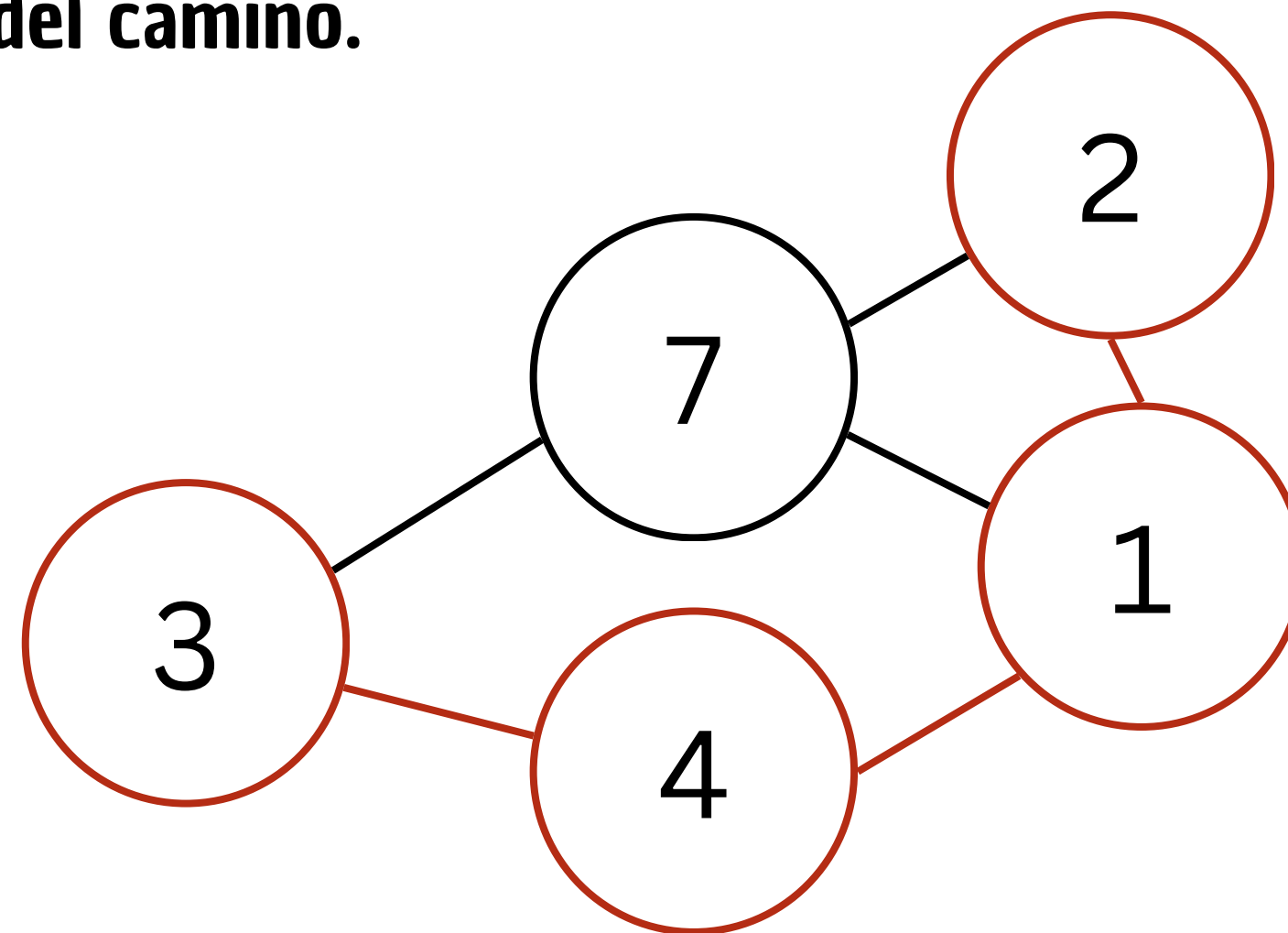
PROBLEMA: PESO EN LOS NODOS

Single-source-shortest-path, con la distancia siendo la suma de los pesos de los nodos del camino.



PROBLEMA: PESO EN LOS NODOS

Single-source-shortest-path, con la distancia siendo la suma de los pesos de los nodos del camino.



PROBLEMA: PESO EN LOS NODOS

Podemos convertir el problema a uno de pesos en las aristas.

